goxpyriment — User Manual

christophe@pallier.org

2026-04-06

Contents

goxpyriment User's Manual

This manual explains the key concepts of the library. It assumes you have read the Getting Started guide and want to understand the framework well enough to write experiments confidently.

Table of Contents

---

1. The Experiment Object

Every experiment revolves around a single `*control.Experiment` value. It is a façade that owns the SDL window, renderer, default font, audio device, keyboard and mouse handlers, and data file. You never create these separately.

```
exp := control.NewExperimentFromFlags("My Experiment", control.Black, control.White, 32
defer exp.End()
```

`NewExperimentFromFlags` parses these optional command-line flags:

Flag	Effect
`-w`	Windowed mode: opens a 1024×768 window instead of going fullscreen
`-d N`	Display ID: open on monitor N (-1 = primary display)
`-s N`	Set subject ID to N (integer); written to the data file automatically
`-headless`	Skip the setup dialog (below) and use field defaults — for batch/automated runs

Automatic setup dialog (launch without `-s`)

If `-s` is **not** passed — most importantly when the binary is launched by **double-clicking its icon** rather than from a terminal — a small setup dialog opens before the experiment starts, letting the experimenter enter the **subject code** and confirm the

display, **fullscreen/windowed** mode, and **results folder**. It seeds its defaults from any `-w`/`-d` you did pass and remembers the display/fullscreen/folder choices across sessions (the subject code is always asked fresh). Cancelling the dialog exits the program.

The dialog is skipped when `-s N` is given (that value is used directly, as before), when `-headless` is passed, or when the program already collected participant information via `GetParticipantInfo` itself — so no program is ever prompted twice.

`defer exp.End()` must appear immediately after construction. It saves the data file, releases fonts, destroys the window, and shuts down SDL — even if the experiment panics or returns early.

Key fields

```
exp.Screen       *apparatus.Screen           // window + renderer
exp.Keyboard     *apparatus.Keyboard         // keyboard input
exp.Mouse        *apparatus.Mouse            // mouse input
exp.AudioDevice  sdl.AudioDeviceID    // SDL audio device for Sound/Tone
exp.Audio        *control.AudioManager // high-level audio helpers
exp.Data         *results.DataFile         // output data file
exp.Design       *design.Experiment   // block/trial structure
exp.SubjectID    int
exp.DefaultFont  *ttf.Font
exp.Info         map[string]string    // values collected by GetParticipantInfo, if use
```

---

2. Collecting Participant Information

Before the experiment window opens, you can display a graphical setup dialog that lets the experimenter fill in participant demographics, monitor characteristics, and display mode. The dialog returns the collected values as a `map[string]string`.

```go
fields := append(control.StandardFields, control.FullscreenField)
info, err := control.GetParticipantInfo("My Experiment", fields)
if err != nil {
    log.Fatalf("setup cancelled: %v", err)
}
```

Call `GetParticipantInfo` **before** `NewExperiment` or `NewExperimentFromFlags`. It initializes SDL internally, shows the window, and shuts SDL down again before returning, so the subsequent experiment initialization starts from a clean state.

Pre-built field sets

Variable	Fields included
`control.ParticipantFields`	`subject_id`, `age`, `gender`, `handedness`

Variable	Fields included
`control.MonitorFields`	`screen_width_cm`, `viewing_distance_cm`, `refresh_rate_hz`
`control.StandardFields`	`ParticipantFields` + `MonitorFields` combined
`control.FullscreenField`	Checkbox: `fullscreen` (`"true"` / `"false"`)

Defining custom fields

```
fields := []control.InfoField{
    {Name: "subject_id", Label: "Subject ID",      Default: ""},
    {Name: "session",    Label: "Session (1/2/3)", Default: "1"},
    {Name: "room",       Label: "Testing room",    Default: "Lab A"},
    {Name: "fullscreen", Label: "Fullscreen mode", Default: "true",
     Type: control.FieldCheckbox},
}
info, err := control.GetParticipantInfo("My Experiment", fields)
```

Fields of type `FieldText` (the default) render as text input boxes. Fields of type `FieldCheckbox` render as tick-boxes; their value is always `"true"` or `"false"`.

Dialog interaction

Action	Effect
Click a field	Focus it
Type	Append text to the focused field
Backspace	Delete last character
Tab / Shift-Tab	Move focus to next / previous text field
Enter	Confirm (same as clicking OK)
Escape / close window	Cancel — `ErrCancelled` is returned

Session persistence

All values except `subject_id` are saved to `~/.cache/goxpyriment/last_session.json` when the experimenter confirms. They are pre-filled on the next run. `subject_id` is always reset — the experimenter must enter it fresh each session.

Using the results

```
info, err := control.GetParticipantInfo("My Experiment", fields)
if err != nil {
    log.Fatalf("setup cancelled: %v", err)
}
```

```go
// Use the fullscreen checkbox to choose the window mode
fullscreen := info["fullscreen"] == "true"
width, height := 0, 0
if !fullscreen {
    width, height = 1024, 768
}

exp := control.NewExperiment("My Experiment", width, height, fullscreen,
    control.Black, control.White, 32)

// Set Info and SubjectID BEFORE Initialize so they are written to the .csv header
exp.SubjectID, _ = strconv.Atoi(info["subject_id"])
exp.Info = info

if err := exp.Initialize(); err != nil {
    log.Fatalf("failed to initialize: %v", err)
}
defer exp.End()
```

When `exp.Info` is non-nil at the time `Initialize()` is called, the collected key/value pairs are automatically written as a `--PARTICIPANT INFO` block in the `.csv` metadata header — no extra call is required.

> **Note:** When using `GetParticipantInfo` you will typically call the lower-level `NewExperiment` + `Initialize()` instead of `NewExperimentFromFlags`, so you can pass the fullscreen/windowed choice from the dialog directly.

---

3. The Run Loop and Error Handling

The `exp.Run` wrapper

All experiment logic runs inside a callback passed to `exp.Run`:

```go
err := exp.Run(func() error {
    // your experiment here
    return control.EndLoop
})
if err != nil && !control.IsEndLoop(err) {
    log.Fatalf("experiment error: %v", err)
}
```

`exp.Run` does two important things:

1. **Ensures everything runs on the SDL main thread.** SDL requires that all rendering and event calls happen on the thread that created the window. `exp.Run` guarantees this.
2. **Installs a panic/recover guard.** When the participant presses ESC, the library immediately aborts the trial loop by calling `panic` with an internal sentinel value.

`exp.Run` catches this panic, converts it back to an error, and returns it cleanly. Your data is saved.

Returning from the callback

Return value	Meaning
`control.EndLoop`	Normal exit; experiment is done
`nil`	Continue — call the callback again on the next frame
any other `error`	Propagated as the return value of `exp.Run`

In a typical experiment you run the full trial loop inside a single callback invocation and return `control.EndLoop` at the end. The `nil` / "keep looping" pattern is used for frame-by-frame animation; in most cases you will never return `nil`.

You don't need to check every error

Because pressing ESC triggers the panic/recover mechanism, any call that would have returned an error (e.g., `exp.Show`, `exp.Wait`, `exp.Keyboard.WaitKey`) will instead abort the loop immediately. The error never reaches your code.

This means the following two styles are both correct:

```
// Style A — check errors explicitly (safer for library/production code)
if err := exp.Show(fixation); err != nil {
    return err
}
if err := exp.Wait(500); err != nil {
    return err
}

// Style B — omit error checks inside exp.Run (fine for experiment scripts)
exp.ShowTimed(fixation, 500)
```

Style B works because if something goes wrong (ESC pressed, window closed), the panic/recover mechanism unwinds the call stack before your code can observe an error. Use Style A if you have cleanup that must run on abort; otherwise Style B keeps experiment scripts readable.

---

4. The Coordinate System

All stimulus positions use a **center-origin** system:

```
            +Y (up)
             |
   -X --------+-------- +X
             |
            -Y (down)
```

- `(0, 0)` is the screen center.
- Positive Y is **up** (opposite to SDL's default, which is top-down).
- Units are pixels at the logical resolution.

```
// Center of screen
stimuli.NewTextLine("Hello", 0, 0, control.White)

// 200 px to the right
stimuli.NewCircle(30, control.Red).SetPosition(control.Point(200, 0))

// Bottom-left area
stimuli.NewTextLine("Score", -400, -250, control.White)
```

The conversion to SDL coordinates (top-left origin) is handled internally by `screen.CenterToSDL(x, y)`. You never call this yourself unless you are writing a custom stimulus type — in which case `screen.CenteredRect(pos, w, h)` returns the SDL destination rectangle for a `w×h` texture centered at `pos`, factoring out the convert-and-offset step.

Logical size

If you call `exp.SetLogicalSize(width, height)`, the coordinate system scales to that virtual resolution regardless of the actual window or screen size. This is useful for making experiments resolution-independent:

```
if err := exp.SetLogicalSize(1920, 1080); err != nil {
    log.Printf("warning: %v", err)
}
// Now (960, 540) is the center-right area, regardless of actual screen size.
```

Display scaling (HiDPI / fractional scaling)

> **Recommendation: set your OS display scaling to 100% before running experiments.**

Most desktop environments (GNOME, KDE, Windows, macOS) allow the user to scale the entire UI — e.g. 125%, 133%, 150% — to make text and icons larger on high-density screens. Goxpyriment does not currently compensate for fractional OS-level scaling. Running with a scale factor other than 100% can cause:

- stimulus positions and sizes to be wrong (coordinates are in logical pixels, but the OS maps them to physical pixels at the scale factor),
- video playback and canvas rendering to appear cropped or misaligned,
- the window to not fill the screen correctly in fullscreen mode.

To disable scaling before running an experiment:

- **GNOME (Ubuntu):** *Settings → Displays → Scale → 100%*
- **KDE:** *System Settings → Display and Monitor → Scale → 100%*
- **Windows:** *Settings → Display → Scale → 100%*

- **macOS:** *System Settings → Displays → Resolution → Default* (or choose a non-HiDPI mode)

You can restore your preferred scaling after the session.

---

5. The Rendering Model

SDL uses a **double-buffered** rendering model. There is an off-screen backbuffer where you draw, and the visible display. You draw to the backbuffer; calling `screen.Update()` (also called a "flip") presents it to the screen, synchronized to the vertical retrace (VSYNC).

The three-step cycle for showing one stimulus is:

```
exp.Screen.Clear()       // fill backbuffer with background color
myStim.Draw(exp.Screen)  // draw stimulus onto backbuffer
exp.Screen.Update()      // present to display (VSYNC-blocks)
```

`exp.Show(stim)` does all three in one call and is the right choice for single-stimulus presentations. For cases where you need to draw multiple stimuli simultaneously — so they appear on screen at the same time — call each `Draw` separately before the single `Update`:

> **Triple/mailbox buffering — handled for you.** On many systems (NVIDIA + compositor, Wayland, and more besides) `SDL_RenderPresent` does **not** block until the retrace: it queues the frame in a non-blocking mode — mailbox ("triple buffering"), where a newer frame replaces the pending one — or hands the buffer to a compositor that accepts it rather than blocking on scanout. The display stays tear-free, but the *call* no longer paces one frame per call. A per-frame loop then completes without consuming wall-clock time and the stimulus is replaced too early, its on-screen duration unpredictably short (often, but not always, zero frames).
>
> `screen.Update()` therefore presents **and then holds to the frame boundary**, so one call always occupies one display frame. You do not have to select a special method or detect the platform: where the driver blocks correctly the wait runs zero iterations and costs nothing. This is why there is no `PacedFlip` — it was removed once pacing became unconditional.
>
> The one thing pacing cannot do is recover a frame that was *dropped* on the way to the panel: it enforces a minimum frame time, not a maximum. See §Calibration below.

```
// Show fixation cross and stimulus simultaneously
exp.Screen.Clear()
fixation.Draw(exp.Screen)
targetCircle.Draw(exp.Screen)
exp.Screen.Update()
```

The blank screen

`exp.Blank(ms)` clears the screen, flips it (showing a blank), and then waits:

```
exp.Blank(1000) // 1-second blank inter-trial interval
```

This is equivalent to `exp.Screen.Clear() + exp.Screen.Update() + exp.Wait(ms)`.

Never draw outside exp.Run

All rendering must happen inside the `exp.Run` callback — equivalently, on the SDL main thread. Drawing from a goroutine will silently do nothing or crash.

---

6. Timing Architecture

Frame cadence

`screen.Update()` returns at the display's vertical retrace (VSYNC). On a 60 Hz monitor this is ~16.67 ms; on a 120 Hz monitor ~8.33 ms. This is the fundamental clock of the visual display: every stimulus change is aligned to a frame boundary automatically.

On some systems (NVIDIA proprietary driver + compositor, Wayland) the system selects a non-blocking presentation mode (mailbox/"triple buffering") or routes the swap through a compositor, so the underlying `SDL_RenderPresent` returns before the frame is scanned out. The display is still tear-free, but the call cannot be relied on to block for one frame. (This is not VSYNC being disabled: a blocking mode such as FIFO exists; the system just isn't guaranteed to use it. Wayland paces via frame callbacks rather than a blocking swap.)

`screen.Update()` closes this gap itself by holding to the frame boundary after presenting, so a tight frame loop is correct on every driver without you choosing anything. Measured on Intel i915 + Wayland with a 120 Hz panel, unaided presents returned as little as 6.95 ms apart against an 8.33 ms frame; through `Update` the same loop held 8.333 ms with SD 0.06 ms.

To know your frame duration at runtime:

```
frameDur := exp.Screen.FrameDuration()  // e.g., 16.666ms on a 60 Hz display
fmt.Printf("Frame: %.2f ms\n", frameDur.Seconds()*1000)
```

Startup calibration

`FrameDuration()` is the *nominal* period — what SDL derives from the display mode. It is not proof that your frames actually took that long. `Initialize()` therefore measures the display over 60 frames before the experiment starts and records both numbers in the `-info.txt` file:

```
# sys refresh_nominal_hz: 120.0000
# sys refresh_measured_hz: 119.9820
```

If the two disagree by more than 10%, a warning is logged. Take it seriously — it means this session's stimulus durations were not what your analysis will assume. The two directions mean different things:

Measured vs nominal	Meaning	Does pacing help?
≈ **nominal**	Normal.	—
Faster than nominal	`SDL_RenderPresent` is not blocking to the retrace (triple/mailbox buffering).	Yes — `Update`'s pacing is exactly this case.
Slower than nominal	Frames are being *dropped* before reaching the panel: a compositor throttling an unfocused or occluded window, or a GPU that cannot keep up.	**No.** Pacing enforces a minimum frame time, not a maximum.

The third row is the one that catches people out. A compositor will happily throttle a windowed experiment to a fraction of the refresh rate while every timestamp your program records still looks plausible. Run fullscreen, keep the window focused, and check the measured rate in the info file afterwards.

To measure it yourself at any point — it bypasses the pacing, so it reports the unaided driver behaviour:

```
measured, err := exp.Screen.CalibrateRefresh(120)  // median over 120 frames
```

`tests/test_vsync_blocking` reports nominal, unaided and paced side by side, with a verdict; run it once on any machine you plan to collect data on.

`exp.Wait` vs `clock.Wait`

Function	Pumps SDL events?	Detects ESC?	Use when
`exp.Wait(ms)`	Yes	Yes	Inside `exp.Run`, between stimuli
`clock.Wait(ms)`	No	No	Short busy-waits, inside animation loops

Always use `exp.Wait` for inter-trial and inter-stimulus intervals — it keeps the OS responsive and responds to ESC. Use `clock.Wait` only inside VSYNC-locked loops that handle events themselves (streams, animation loops).

Sub-millisecond timing is not guaranteed for `exp.Wait`

`exp.Wait` sleeps in 1 ms increments. For coarse delays (ISIs, fixation durations) this is perfectly fine. For frame-accurate stimulus timing — e.g., showing a stimulus for

exactly 2 frames — use the stream functions described in Section 10.

Nanosecond event timestamps

SDL3 timestamps every input event **at hardware-interrupt time**, before any application code runs. The timestamp is stored with the event in the SDL event queue, where it remains untouched until your code reads it.

This has a crucial consequence: **it does not matter when you call `GetKeyEventTS`**. Any keypress that occurred — even during `exp.Wait`, between two `ShowTS` calls, or while other computation was running — is waiting in the queue with its exact hardware timestamp. Nothing is lost. This is fundamentally different from a polling-based approach, where a keypress can only be measured from the moment the polling function is called, and any code running before that call inflates the apparent RT.

`exp.ShowTS(stim)` records the SDL nanosecond time of the VSYNC flip. `GetKeyEventTS` retrieves the event's own hardware timestamp. Subtracting the two gives hardware-precision RT regardless of how much code ran in between:

```
onset, _ := exp.ShowTS(stim1)   // record flip timestamp
exp.Wait(500)                    // keypress here is NOT lost
exp.ShowTS(stim2)                // keypress here is also NOT lost
key, keyTS, _ := exp.Keyboard.GetKeyEventTS(responseKeys, -1)
rtToStim1 := int64(keyTS - onset)   // correct even if pressed during Wait
```

This is especially useful when you need RT relative to a specific stimulus in a multi-stimulus sequence — something that `WaitKeysRT` cannot express directly, since it measures from the call site rather than from a recorded onset.

Two clocks: the Go clock and the SDL clock

goxpyriment exposes time through **two independent clocks with different zero points**. A value read from one must never be subtracted from a value read from the other — the difference would silently include the unknown, machine-dependent offset between their origins.

Clock	Read it via	Zero point	Use it for
Go monotonic clock	`clock.GetTime()`, `clock.GetTimeNS()`, `Clock.Now()`, `Clock.NowMillis()`, `Clock.NowNanos()`	First use of the `clock` package, or `clock.NewClock()`	*Scheduling and logging in your own code*: inter-trial and inter-stimulus intervals, drift-free trial pacing (`SleepUntil`), and "time since start of block" columns in your data file

Clock	Read it via	Zero point	Use it for
SDL event clock	`exp.ShowTS()`, `Screen.FlipTS()`, `Keyboard.GetKeyEventTS()`, `WaitAnyEventTS()`, and the `Timestamp` field of any SDL event (all on `sdl.TicksNS()`)	SDL initialisation	**Reaction-time measurement**: whenever you subtract a stimulus onset from a response, *both* values must come from this clock

The rule. Measure reaction times entirely on the SDL clock — subtract a `FlipTS`/`ShowTS` onset from a `GetKeyEventTS` response. Use the Go clock only for things you *schedule* rather than *measure*: ISIs, fixation durations, block pacing, and log timestamps. Crossing the two — e.g. `keyTS - clock.GetTimeNS()` — is meaningless.

Why two clocks? The SDL clock is the one SDL stamps onto hardware input events at interrupt time (see the previous subsection), so it is the only clock that can measure RT without inflation from intervening code. The Go clock needs no SDL context, is what `exp.Wait`, `clock.Wait`, and `Clock.SleepUntil` are built on, and is the natural choice for scheduling and for log timestamps that just need to be internally consistent.

The two roles coexist cleanly in a single trial loop — Go clock for pacing and logging, SDL clock for the RT:

```
c := clock.NewClock()                                    // Go clock: pacing + logging
for i, trial := range block.Trials {
    c.SleepUntil(time.Duration(i) * 2 * time.Second)   // Go clock: drift-free onset ca
    onset, _ := exp.ShowTS(trial.Stimulus)              // SDL clock: stimulus onset
    key, keyTS, _ := exp.Keyboard.GetKeyEventTS(responseKeys, -1)
    rtMs := float64(keyTS-onset) / 1e6                   // SDL − SDL  ✓ valid RT
    exp.Data.Add(trial.Label, key, rtMs, c.NowMillis())// log time on the Go clock
}
```

Here `keyTS - onset` stays within the SDL clock (a real RT), while `c.SleepUntil` and `c.NowMillis()` stay within the Go clock (scheduling and a log column). The two are never subtracted from each other.

Disabling garbage collection

Go's garbage collector can pause execution for several milliseconds at unpredictable times. The stream and animation functions disable it during the critical loop:

```
old := debug.SetGCPercent(-1)
defer debug.SetGCPercent(old)
```

You do not need to do this yourself for ordinary trial loops; only for high-speed RSVP or animation. The stream and animation functions handle it automatically.

Choosing a timing approach

Use this decision guide before committing to a timing strategy.

Situation	Recommended approach
Coarse ISIs and fixation durations ($\geq$ 100 ms)	`exp.Wait(ms)` — adequate precision, keeps the event loop alive
Show stimulus + timed hold (fixation, cue, mask)	`exp.ShowTimed(stim, ms)` — `Show` + `Wait` in one call
Single-stimulus trial, RT relative to onset	`key, rt, err := exp.ShowAndGetRT(stim, keys, timeout)` — hardware-precise RT, clears stale events automatically
Stimulus duration measured in frames (e.g. 2-frame mask)	Per-frame loop with `screen.Flip()` (paced, safe on all drivers), or `PresentStreamOfImages` (Section 10)
Multi-stimulus sequence, RT relative to a specific stimulus	`exp.ShowTS(stim)` + `exp.Keyboard.GetKeyEventTS(...)` — nanosecond timestamps on the same SDL clock; subtract directly
RSVP or rapid animation (frame-accurate presentation of many items)	`PresentStreamOfImages` / `PresentStreamOfText` (Section 10) — GC disabled, VSYNC-locked, full timing log
EEG/MEG synchronisation required	Add a TTL pulse via `triggers` immediately after `exp.ShowTS` (Section 17)
Stimulus duration not a multiple of the frame period (e.g. 10 ms, 17 ms)	Variable Refresh Rate mode — see below

OS considerations. Presentation latency depends heavily on whether the display compositor is active — see the next section. For EEG work, always verify onset timing with a photodiode regardless of OS.

When in doubt, use `ShowAndGetRT`. It clears stale events, records the VSYNC flip timestamp, waits for the key with hardware-precision timing, and returns `(key, rtMs, error)` — the canonical single-stimulus RT call. Use raw `ShowTS` + `GetKeyEventTS` only when composing multiple stimuli before a single response.

`ShowTS` vs `FlipTS`. `exp.ShowTS(stim)` is the standard high-level call: it clears the screen, draws the stimulus, flips, and returns the flip timestamp — one line for the common case. `exp.Screen.FlipTS()` is the lower-level primitive that only flips and timestamps, leaving clearing and drawing to you. Both hold to the frame boundary, so both are safe in a per-frame loop; there is no third variant to choose between.

Display compositor bypass

A compositing window manager blends every application frame with other on-screen elements before sending the result to the display. This adds latency and jitter that can

be significant for millisecond-accurate stimulus presentation. How much overhead you incur — and whether you can avoid it — depends on the operating system.

Linux / X11 (fullscreen). SDL3 automatically sets the `_NET_WM_BYPASS_COMPOSITOR` window property when the application enters fullscreen mode. Compliant compositing WMs (KWin, Mutter, Compton) respond by *unredirecting* the fullscreen window: the application's framebuffer is routed directly to the display scan-out pipeline without compositing. Presentation latency drops below 1 ms — comparable to a compositor-free session. Nothing special needs to be done in goxpyriment; running fullscreen (omit `-w`) is sufficient.

Linux / Wayland (fullscreen). The Wayland compositor is always the intermediary for frame delivery, but modern compositors (KWin 5.21+, GNOME Mutter 44+) support *direct scan-out* for fullscreen applications, which routes the framebuffer to the display hardware with near-zero overhead. In practice the latency is similar to X11 bypass, though this depends on the compositor version and GPU driver.

Linux / KMS–DRM (no display server). The lowest-latency configuration on Linux is to run without any display server at all, from a virtual terminal (`Ctrl+Alt+F2`). Setting the environment variable `SDL_VIDEODRIVER=kmsdrm` before launching the experiment directs SDL3 to use the Linux kernel's KMS/DRM subsystem directly, bypassing both X11 and Wayland entirely:

```
SDL_VIDEODRIVER=kmsdrm go run myexperiment/main.go
```

This is the recommended configuration for the most demanding timing requirements, such as single-frame subliminal stimulation or high-frequency RSVP.

Windows (fullscreen). SDL3's fullscreen mode creates an exclusive fullscreen surface that bypasses the Desktop Window Manager (DWM), yielding frame-interval jitter typically below 0.3 ms (one standard deviation), comparable to Linux.

macOS. Apple's Metal pipeline always delivers frames through the WindowServer compositor, and no third-party application can bypass it — even in fullscreen.

These platform differences can be measured empirically with the `display` and `av` subtests of the bundled `Timing-Tests` suite (see TimingTests.md).

Variable Refresh Rate (VRR) — arbitrary stimulus durations

On a standard 60 Hz monitor, every stimulus duration must be a multiple of 16.67 ms. You cannot present a stimulus for 10 ms or 20 ms — the display simply cannot change in between frame boundaries. Variable Refresh Rate technology (AMD FreeSync, NVIDIA G-Sync, VESA Adaptive-Sync) removes this constraint: the panel holds each frame for exactly as long as the software asks, refreshing only when the next `Present()` call arrives.

Enabling VRR in goxpyriment goxpyriment opens the renderer with VSYNC enabled by default, which locks every flip to a frame boundary. To enter VRR mode, disable VSYNC before the timing-critical section and restore it afterwards:

```
// Switch to VRR mode — Present() now returns immediately.
if err := exp.Screen.SetVSync(0); err != nil {
    log.Fatalf("VRR not supported: %v", err)
}
defer exp.Screen.SetVSync(1) // always restore

// Pre-load stimulus textures before disabling GC.
stimuli.PreloadVisualOnScreen(exp.Screen, myStim)

old := debug.SetGCPercent(-1)
defer debug.SetGCPercent(old)

// Draw stimulus and present (returns immediately with VSync=0).
exp.Screen.Renderer.SetDrawColor(0, 0, 0, 255)
exp.Screen.Clear()
myStim.Draw(exp.Screen)
onsetNS, _ := exp.Screen.FlipTS()

// Hold for exactly targetDur using a busy-wait (sub-ms precision).
deadline := time.Now().Add(targetDur)
for time.Now().Before(deadline) { /* spin */ }

// Present blank — the display switches immediately on VRR.
exp.Screen.Renderer.SetDrawColor(0, 0, 0, 255)
exp.Screen.Clear()
offsetNS, _ := exp.Screen.FlipTS()

actualDurMs := float64(offsetNS-onsetNS) / 1e6
```

`onsetNS` and `offsetNS` are both `sdl.TicksNS()` timestamps on the SDL nanosecond clock, captured right after each `Present()` returns. Their difference measures the software-controlled stimulus duration. On a VRR display this equals the actual on-screen duration plus a small, constant hardware pipeline latency (GPU scan-out + panel response, typically 1–5 ms) that can be measured independently with the `frames` test and a photodiode.

VRR range and self-diagnosis Every VRR panel has a supported refresh-rate range (e.g. 48–144 Hz on a typical gaming monitor). Requesting a duration shorter than `1000/max_Hz` ms or longer than `1000/min_Hz` ms takes the panel outside its VRR window; the display then falls back to fixed-rate behaviour and durations are again quantised to frame multiples.

You can characterise your panel's VRR window empirically with the `Timing-Tests` suite:

```
go run ./tests/Timing-Tests -test vrr -vrr-max-ms 50 -cycles 5
```

This sweeps target durations from 1 ms to 50 ms in 1 ms steps and reports the ac-

tual vs. target duration at each step. Duration errors below 0.5 ms across the sweep confirm that VRR is working; large periodic errors confirm that VRR is absent or the requested duration is outside the supported range. See TimingTests.md — VRR section for detailed interpretation, including how to read the VRR boundary from the output and how to enable FreeSync on Linux.

Requirements

- A VRR-capable monitor (FreeSync, G-Sync Compatible, or Adaptive-Sync).
- VRR enabled in the OS display settings (Linux: DRM/KMS or compositor; Windows: Display Settings → Advanced display → Variable refresh rate).
- The application does not need any special SDL hints; `SetVSync(0)` is sufficient.

Note on tearing On a monitor without VRR, `SetVSync(0)` disables the frame-boundary synchronisation entirely and the GPU writes to the framebuffer while the display is reading it, producing a horizontal tear line. On a VRR monitor this does not happen: the display waits for the GPU's signal before starting each new refresh. If your VRR test shows tearing artefacts, VRR is either not enabled in your OS or not supported by your hardware.

---

7. Input Handling

Keyboard

The `exp.Keyboard` object provides blocking and non-blocking access:

```
// Block until any key — returns the keycode
key, err := exp.Keyboard.Wait()

// Block until a specific key
err := exp.Keyboard.WaitKey(control.K_SPACE)

// Block until one of several keys, with optional timeout (-1 = no timeout)
// Returns 0 on timeout, sdl.EndLoop on ESC/quit
key, err := exp.Keyboard.WaitKeys([]control.Keycode{control.K_F, control.K_J}, 3000)

// Same, but also returns reaction time in milliseconds from the call site
key, rt, err := exp.Keyboard.WaitKeysRT([]control.Keycode{control.K_F, control.K_J}, 30

// Hardware-precision version: returns the SDL event's nanosecond timestamp
key, eventTS, err := exp.Keyboard.GetKeyEventTS([]control.Keycode{control.K_F, control.

// Non-blocking poll — returns 0 if nothing pressed
key, err := exp.Keyboard.Check()

// Query whether a key is physically held down right now (no event queue involvement)
held := exp.Keyboard.IsPressed(control.K_SPACE)
```

```
// Wait for KEY_UP on a specific key; returns its SDL hardware timestamp (nanoseconds)
upTS, err := exp.Keyboard.WaitKeyReleaseTS(key, -1)

// Drain all pending key events (use before a new trial to discard stale presses)
exp.Keyboard.Clear()
```

`GetKeyEventTS` "get" semantics. Unlike the `Wait*` functions, `GetKeyEventTS` reads from the SDL event queue and returns *immediately* if a matching event is already there — it only blocks when the queue has no matching event. This means a keypress that happened during `exp.Wait` or any other intervening code is consumed instantly on the next `GetKeyEventTS` call, with its original hardware timestamp intact.

When to call `Clear()`. Despite living on `exp.Keyboard`, `Clear()` drains the **entire** SDL event queue — keyboard, mouse, gamepad, and everything else — because SDL uses a single shared queue. Call it at the start of a trial (before `ShowTS`) to discard stale events from any device. Do **not** call it after `ShowTS`: the participant may have already responded, and `Clear()` would silently discard that event before `GetKeyEventTS` or `GetPressEventTS` ever sees it.

`WaitKeysRT` vs `GetKeyEventTS`:

`WaitKeysRT` measures elapsed time from the moment the call is made (using `sdl.Ticks()`, millisecond granularity). In a single-stimulus trial this is a good approximation of RT from stimulus onset, but it accumulates any delay between `Show()` returning and `WaitKeysRT` being called.

`GetKeyEventTS` returns the SDL3 event's own `Timestamp` field — the nanosecond time at which the hardware key-down event was generated. Combined with `exp.ShowTS(stim)`, which records the nanosecond time of the VSYNC flip, reaction time is simply:

```
key, rtMs, _ := exp.ShowAndGetRT(target, responseKeys, 3000)
```

Internally `ShowAndGetRT` calls `exp.Keyboard.Clear()`, then `exp.ShowTS` to get the VSYNC flip timestamp, then `GetKeyEventTS`, and returns the difference in milliseconds. When you need the raw nanosecond timestamps (e.g. for EEG trigger alignment), you can call the primitives directly:

```
onset, _ := exp.ShowTS(target)   // nanoseconds at VSYNC flip
key, eventTS, _ := exp.Keyboard.GetKeyEventTS(responseKeys, 3000)
rtNS := int64(eventTS - onset)   // nanoseconds; divide by 1e6 for ms
```

This form is also the only correct approach when multiple stimuli are presented in sequence and you need RT relative to a specific one:

```
onset1, _ := exp.ShowTS(prime)   // prime appears
exp.Wait(500)
exp.ShowTS(target)               // target appears 500 ms later
key, eventTS, _ := exp.Keyboard.GetKeyEventTS(responseKeys, 3000)
rtToPrime := int64(eventTS - onset1)  // RT from prime onset
```

Collecting multiple simultaneous responses — `GetKeyEventsTS`:

In rare cases you may want to capture *all* key events, not just the first. `GetKeyEventsTS` is designed for detecting bilateral responses (e.g. both hands pressing at once) and works in two phases:

1. **Phase 1** — blocks until the first matching key arrives, respecting the timeout exactly like `GetKeyEventTS`.
2. **Phase 2** — waits up to 50 ms for any additional matching keys.

The second phase is necessary because human "simultaneous" presses are rarely truly simultaneous — the two KEY_DOWN events typically arrive 10–50 ms apart. Without this window, a non-blocking drain after the first key would miss the second key almost every time.

```
events, err := exp.Keyboard.GetKeyEventsTS(responseKeys, 3000)
if len(events) > 0 {
    firstKey := events[0].Key
    firstTS  := events[0].TimestampNS
    rtNS     := int64(firstTS - onset)
}
if len(events) == 2 {
    lagNS := int64(events[1].TimestampNS - events[0].TimestampNS)
}
```

In the common single-response case only one event is returned, at the cost of at most 50 ms of extra latency before the function returns. For the typical single-response trial, `GetKeyEventTS` avoids that overhead entirely.

Recording all presses over a fixed duration — `CollectKeyEventsTS`:

When the goal is to record *everything* the participant presses within a known time window (finger-tapping, free-response periods, stimulus-stream logging), use `CollectKeyEventsTS`. Unlike `GetKeyEventsTS`, it always runs for the full duration — it does not return early on the first keypress:

```
// Record every tap of F or J over 5 seconds
exp.Keyboard.Clear()
onset, _ := exp.ShowTS(stim)
events, err := exp.Keyboard.CollectKeyEventsTS(
    []control.Keycode{control.K_F, control.K_J}, 5000)
for _, ev := range events {
    rtNS := int64(ev.TimestampNS - onset)
    fmt.Printf("key %s at %d ms\n", ev.Key.KeyName(), rtNS/1_000_000)
}
```

Pass `keys = nil` to capture any key. Pass `durationMS = 0` to do a non-blocking drain of whatever is already in the queue. Returns an empty (non-nil) slice if nothing was pressed.

Querying whether a key is currently held — `IsPressed`:

`IsPressed` queries SDL's internal scancode state array and returns `true` if the key is

physically depressed at the instant of the call, regardless of the event queue. It is useful in animation loops that need to react continuously to a held key, or to verify that the participant has released a key before starting a new trial:

```
for exp.Keyboard.IsPressed(control.K_SPACE) {
    // do something while SPACE is held
    time.Sleep(10 * time.Millisecond)
}
```

Because `IsPressed` does not consume queue events, it can be called freely alongside `GetKeyEventTS` without interfering with the event stream.

Measuring keypress duration — `WaitKeyReleaseTS`:

`WaitKeyReleaseTS` blocks until a KEY_UP event arrives for the specified key and returns its SDL3 hardware timestamp in nanoseconds. Together with the KEY_DOWN timestamp from `GetKeyEventTS`, this gives nanosecond-precision keypress duration:

```
key, downTS, _ := exp.Keyboard.GetKeyEventTS(responseKeys, -1)
// ... optionally update display while key is held ...
upTS, _ := exp.Keyboard.WaitKeyReleaseTS(key, 5000)
durationNS := upTS - downTS           // nanoseconds
durationMS := durationNS / 1_000_000  // milliseconds
```

This is the recommended approach for paradigms where press duration is a dependent variable (e.g., force-choice hold-to-respond, finger-tapping, or hold-triggered responses).

Mouse

```
// Block until any button
btn, err := exp.Mouse.WaitPress()

// RT in milliseconds from call site (mirrors Keyboard.WaitKeysRT)
btn, rt, err := exp.Mouse.WaitPressRT(3000)

// Hardware-precision version: returns the SDL event's nanosecond timestamp
btn, eventTS, err := exp.Mouse.GetPressEventTS(3000)

// Non-blocking poll
btn, err := exp.Mouse.Check()

// Query whether a button is physically held down right now
held := exp.Mouse.IsPressed(sdl.BUTTON_LEFT)

// Wait for MOUSE_BUTTON_UP; returns its SDL hardware timestamp (nanoseconds)
upTS, err := exp.Mouse.WaitButtonReleaseTS(btn, 5000)

// Current cursor position in center-based coordinates
x, y := exp.Mouse.Position()
```

```
// Show / hide cursor
exp.Mouse.ShowCursor(false)
```

The cursor is hidden by default. `Initialize()` (and therefore `NewExperimentFromFlags`) hides the mouse pointer, so it never sits on top of your stimuli. Mouse-driven paradigms must ask for it back, right after the experiment is created:

```
exp := control.NewExperimentFromFlags("Mouse Tracking", control.Black, control.White, 3
defer exp.End()
exp.ShowCursor()   // participants need to see what they are pointing at
```

Setting `exp.CursorVisible = true` before `Initialize()` does the same thing. The participant-info dialog always shows the cursor while it is open, whatever the experiment's setting, since it is clicked.

Button values: `sdl.BUTTON_LEFT`, `sdl.BUTTON_MIDDLE`, `sdl.BUTTON_RIGHT`, `sdl.BUTTON_X1`, `sdl.BUTTON_X2`.

`IsPressed` and `WaitButtonReleaseTS` mirror the keyboard's `IsPressed` and `WaitKeyReleaseTS` and can be used to measure click-hold duration:

```
btn, downTS, _ := exp.Mouse.GetPressEventTS(-1)
upTS, _        := exp.Mouse.WaitButtonReleaseTS(btn, 5000)
durationMS     := int64(upTS-downTS) / 1_000_000
```

Multi-device input — `WaitAnyEventTS`

If you want to accept a response from *either* the keyboard or the mouse (or both), use `exp.WaitAnyEventTS` instead of calling `Keyboard` and `Mouse` separately:

```
onset, _ := exp.ShowTS(stim)

// Accept F or J key, or any mouse button click, up to 3 s
ev, err := exp.WaitAnyEventTS(
    []control.Keycode{control.K_F, control.K_J},
    true,   // catchMouse
    3000,
)

rtNS := int64(ev.TimestampNS - onset)  // hardware-precision RT, nanoseconds
rtMs := rtNS / 1_000_000

switch ev.Device {
case apparatus.DeviceKeyboard:
    fmt.Printf("key %d, RT %d ms\n", ev.Key, rtMs)
case apparatus.DeviceMouse:
    fmt.Printf("mouse button %d, RT %d ms\n", ev.Button, rtMs)
}
```

Pass `keys = nil` to accept any key. Pass `catchMouse = false` to ignore the mouse. On

timeout, returns a zero `InputEvent` with `nil` error.

Key codes

Key codes are re-exported in `control` so experiment code never needs to import `go-sdl3` just to name a key. Coverage includes the full alphabet (`K_A` … `K_Z`), the digit row (`K_0` … `K_9`), the numeric keypad (`K_KP_0` … `K_KP_9`, `K_KP_ENTER`, `K_KP_PLUS`, `K_KP_MINUS`), navigation/control keys (`K_SPACE`, `K_ESCAPE`, `K_RETURN`, `K_BACKSPACE`, `K_TAB`, the arrow keys), and common punctuation (`K_MINUS`, `K_PLUS`, `K_EQUALS`, `K_LEFTBRACKET`, `K_RIGHTBRACKET`):

```
key, _ := exp.Keyboard.Wait()
if key == control.K_A { ... }
```

For a rarely-used code not in `defaults.go`, fall back to `go-sdl3/sdl` directly (e.g. `sdl.K_SEMICOLON`).

Response Devices

All of the input methods above are device-specific: `GetKeyEventTS` for keyboards, `GetPressEventTS` for the mouse, and so on. If the experiment should run equally well with different input hardware — a keyboard in one lab, a response box in another — device-specific calls scatter conditional logic throughout the trial loop.

`ResponseDevice` is a single interface that abstracts over all of them:

```
// in package io
type ResponseDevice interface {
    WaitResponse(ctx context.Context) (Response, error)
    DrainResponses(ctx context.Context) error
    Close() error
}
```

`WaitResponse` blocks until a response is detected and returns a `Response`:

```
type Response struct {
    Source  apparatus.DeviceKind  // which device fired
    Code    uint32         // key code, button index, or TTL bitmask
    RT      time.Duration  // elapsed from WaitResponse call
    Precise bool           // timing quality (see below)
}
```

Wrap any input device with the matching adapter:

```
// Keyboard (SDL hardware timestamps → Precise: true)
var rd apparatus.ResponseDevice = &apparatus.KeyboardResponseDevice{KB: exp.Keyboard}

// Mouse (SDL hardware timestamps → Precise: true)
var rd apparatus.ResponseDevice = &apparatus.MouseResponseDevice{M: exp.Mouse}

// TTL response box, e.g. MEGTTLBox (software poll → Precise: false)
```

```
box, _ := triggers.NewMEGTTLBox("/dev/ttyACM0")
var rd apparatus.ResponseDevice = apparatus.NewTTLResponseDevice(box, 5*time.Millisecon
```

All three look identical inside a trial loop:

```
onset, _ := exp.ShowTS(stim)
_ = rd.DrainResponses(ctx)          // clear stale presses from previous trial
resp, err := rd.WaitResponse(ctx)
if err != nil { /* ESC, timeout, etc. */ }

rtMs := resp.RT.Milliseconds()      // always valid
```

Timing precision and the `Precise` flag The `Precise` field tells you whether `RT` came from a nanosecond hardware event timestamp or a software poll:

Device	`Precise`	RT accuracy
Keyboard, Mouse, Gamepad	`true`	SDL3 hardware event timestamp, nanosecond resolution — identical to using `GetKeyEventTS` directly
MEGTTLBox, DLPIO8	`false`	`time.Now()` at poll detection; accuracy bounded by poll interval (default 5 ms)

When `Precise` is `true`, the RT in the `Response` is computed from `sdl.TicksNS()` captured at the `WaitResponse` call versus the SDL3 hardware event timestamp — the same nanosecond clock used by `Screen.FlipTS`. It is therefore suitable for stimulus-onset-locked RT:

```
onset, _ := exp.ShowTS(stim)
resp, _ := rd.WaitResponse(ctx)
if resp.Precise {
    // resp.RT was computed from SDL hardware timestamps: nanosecond accuracy
    rtFromOnset := time.Duration(int64(onset) + resp.RT.Nanoseconds())  // approximate
}
```

When `Precise` is `false`, `RT` is still a valid elapsed duration — it just has ~poll-interval jitter (5 ms by default). For typical behavioral RT tasks this is acceptable; for EEG/MEG experiments requiring sub-millisecond synchrony, use the TTL output path for stimulus marking and treat RT as an approximate measure or use a hardware response box with sub-ms timestamping.

---

8. Data Collection

Output files

When `exp.End()` is called, two files are written to `~/goxpy_data/`:

File	Contents
`<expName>_sub-<NNN>_date-<YYYYMMDD>-<HHMMSS>.csv`	Pure CSV data rows — directly importable by Excel, R, or pandas
`<expName>_sub-<NNN>_date-<YYYYMMDD>-<HHMMSS>-info.txt`	Session metadata: start/end time, hostname, framework version, participant info, the machine and OS it ran on (model, kernel, desktop/compositor, CPU, every GPU and its driver, sound server and version), and the display and audio configuration SDL actually opened

Both files open automatically on `Initialize()` and are flushed to disk when `exp.End()` is called. Any spaces in `<expName>` are replaced by `-` so the resulting filenames never contain whitespace.

Choosing the output directory

By default, data files are written to `~/goxpy_data/` (the folder name is the `results.DataFileDirectory` constant, appended to the user's home directory). To write them elsewhere, call `exp.SetOutputDirectory()` **before** `Initialize()` opens the files:

```
exp := control.NewExperiment("My Experiment", control.Black, control.White, 32)
exp.SetOutputDirectory("/path/to/mydata")  // must come before Initialize()
if err := exp.Initialize(); err != nil { log.Fatal(err) }
defer exp.End()
```

The directory is created if it does not exist.

Because `NewExperimentFromFlags` calls `Initialize()` for you, overriding the directory requires the lower-level `NewExperiment(...) + Initialize()` path shown above rather than `NewExperimentFromFlags`.

Declaring column names

Call this once, before the trial loop:

```
exp.AddDataVariableNames([]string{"condition", "response", "rt_ms", "correct"})
```

Subject ID is always prepended automatically — do not include it in the name list.

Adding rows

```
exp.Data.Add(condition, response, rt, correct)
```

Fields are written in the same order as the variable names. Any type that has a meaningful `fmt.Sprint` representation works: `int`, `float64`, `bool`, `string`. Fields containing commas or quotes are escaped per RFC 4180.

Example CSV output

```
subject_id,condition,response,rt_ms,correct
3,"congruent","F",412,true
3,"incongruent","J",538,false
```

Numbers and booleans are unquoted; strings are always double-quoted.

Saving mid-experiment

For long experiments it is good practice to call `exp.Data.Save()` after each block. This flushes both files to disk so that data up to that point is not lost if the experiment crashes.

---

9. Stimuli: Lifecycle and Preloading

GPU textures are lazily allocated

Creating a stimulus (e.g., `stimuli.NewTextLine(...)`) is cheap — it allocates a Go struct and stores the parameters, but does no GPU work. The SDL texture is created on the **first call to `Draw`**. This means:

- You can safely create all your stimuli before the run loop.
- The first presentation of each stimulus will be slightly slower due to texture upload.

For timing-sensitive presentations, preload explicitly:

```
stimuli.PreloadVisualOnScreen(exp.Screen, myStim)
// or, for a slice:
stimuli.PreloadAllVisual(exp.Screen, []stimuli.VisualStimulus{stim1, stim2, stim3})
```

Call this after `exp.Run` starts (the renderer is ready), but before the critical section:

```
err := exp.Run(func() error {
    // Preload everything while showing instructions
    exp.ShowInstructions("Loading, please wait...")
    stimuli.PreloadAllVisual(exp.Screen, stimSlice)

    // Now timing-sensitive trials
    for _, stim := range stimSlice { ... }
    return control.EndLoop
})
```

Releasing textures

If you generate many stimuli dynamically (e.g., hundreds of unique text strings), call `stim.Unload()` after each trial to free the GPU memory:

```
for _, word := range wordList {
    stim := stimuli.NewTextLine(word, 0, 0, control.White)
    exp.Show(stim)
    exp.Keyboard.Wait()
    stim.Unload()  // release GPU texture
}
```

For a fixed, small set of stimuli created once and reused throughout the experiment, you never need to call `Unload` — `exp.End()` handles cleanup.

Writing a custom stimulus

Any type that implements `VisualStimulus` can be passed to `exp.Show`:

```
type VisualStimulus interface {
    Present(screen *apparatus.Screen, clear, update bool) error
    Preload() error
    Unload() error
    Draw(screen *apparatus.Screen) error
    GetPosition() sdl.FPoint
    SetPosition(pos sdl.FPoint)
}
```

Embed `stimuli.BaseVisual` to get the position methods and no-op `Preload`/`Unload` for free, then implement only `Draw` and `Present`:

```
type MyStimulus struct {
    stimuli.BaseVisual
    // your fields
}

func (m *MyStimulus) Draw(screen *apparatus.Screen) error {
    // draw using screen.Renderer SDL calls
    return nil
}

func (m *MyStimulus) Present(screen *apparatus.Screen, clear, update bool) error {
    return stimuli.PresentDrawable(m, screen, clear, update)
}
```

Interactive widgets

Two stimuli present a UI and collect a structured response from the participant in a blocking loop. Neither is timing-critical, so they use `sdl.WaitEvent` rather than VSYNC polling.

`Menu` — a numbered, keyboard-navigable list:

```
m := stimuli.NewMenu([]string{"Beginner", "Expert", "Practice run"})
// Optional customisation:
m.HighlightColor = sdl.Color{R: 255, G: 220, B: 0, A: 255}

idx, err := m.Get(exp.Screen, exp.Keyboard, 0)
// idx is 0-based; -1 + sdl.EndLoop on ESC/quit
```

The selected item is shown with a `>` prefix in `HighlightColor`; all others use `TextColor`. Navigation: UP/DOWN arrows move the highlight; ENTER or SPACE confirms; digit keys 1–9 (0 for tenth) select directly without requiring confirmation.

`ChoiceGrid` — a grid of labelled buttons, activated by mouse click or matching key:

```
cg := stimuli.NewChoiceGrid([]string{"B","C","D","F","G"}, 3, "Pick 3 letters:")
selections, err := cg.Get(exp.Screen, exp.Keyboard)
// selections is []string in press order
```

---

10. High-Precision Streams (RSVP)

For paradigms that present stimuli at high speed — RSVP, attentional blink, priming — the standard `exp.Show` / `exp.Wait` cycle is not precise enough. The stream functions provide frame-accurate presentation:

- GC is **disabled** for the duration of the stream.
- Every onset and offset is aligned to a **VSYNC boundary**.
- A `TimingLog` records the predicted and actual onset for each stimulus.
- All keyboard and mouse events that occur during the stream are captured.

Text stream

The simplest entry point:

```
words := []string{"CHAIR", "RIVER", "TIGER", "CLOCK", "STONE"}
on  := 150 * time.Millisecond
off :=  50 * time.Millisecond

events, logs, err := stimuli.PresentStreamOfText(
    exp.Screen, words, on, off,
    0, 0,           // center of screen
    control.White,
)
```

Image / mixed stimulus stream

For images or any `VisualStimulus`:

```
stims := []stimuli.VisualStimulus{pic1, fixation, pic2, fixation}

// Regular cadence (all stimuli share the same on/off duration)
elements := stimuli.MakeRegularVisualStream(stims, 100*time.Millisecond, 0)

// Irregular cadence (individual onset times and durations, in ms)
elements, err := stimuli.MakeVisualStream(stims, onsetMs, durationMs)

events, logs, err := stimuli.PresentStreamOfImages(exp.Screen, elements, 0, 0)
```

Reading events from the stream

Each `UserEvent` carries two timestamps:

- `Timestamp` — a `time.Duration` relative to stream start, Go monotonic clock, millisecond precision. Useful for quick inspection.
- `TimestampNS` — the SDL3 hardware event timestamp in nanoseconds, same clock as `Screen.FlipTS`. Use this for sub-millisecond RT computation.

```
for _, ev := range events {
    if ev.Event.Type == sdl.EVENT_KEY_DOWN {
        key := ev.Event.KeyboardEvent().Key
        t   := ev.Timestamp.Milliseconds()  // ms from stream start (coarse)
        fmt.Printf("key %d pressed at %d ms\n", key, t)
    }
}
```

Use `stimuli.FirstKeyPress` to find the first matching key-down event without writing a loop:

```
if ev, ok := stimuli.FirstKeyPress(events, sdl.K_SPACE); ok {
    fmt.Printf("Space pressed at %d ms\n", ev.Timestamp.Milliseconds())
}
```

Both `ev.Timestamp` (Go clock, ms precision) and `ev.TimestampNS` (SDL3 hardware, nanoseconds) are available on the returned `UserEvent`.

Computing RT from stream events

Because `UserEvent.TimestampNS` and `TimingLog.OnsetNS` are on the same SDL nanosecond clock, reaction time from a specific stimulus is exact:

```
for _, ev := range events {
    if ev.Event.Type == sdl.EVENT_KEY_DOWN {
        for _, l := range logs {
            if ev.TimestampNS >= l.OnsetNS && (l.OffsetNS == 0 || ev.TimestampNS < l.Of
                rtNS := int64(ev.TimestampNS - l.OnsetNS)
                fmt.Printf("RT from stimulus %d: %d ms\n", l.Index, rtNS/1_000_000)
            }
        }
```

```
    }
}
```

Reading the timing log

```
for i, l := range logs {
    // Read the authoritative SDL-clock fields, not the Go-clock ActualOnset/
    // ActualOffset diagnostics. Achieved on-screen duration vs target = jitter.
    shownMs := int64(l.OffsetNS-l.OnsetNS) / 1_000_000
    jitter := shownMs - l.TargetOn.Milliseconds()
    fmt.Printf("stimulus %d: target %d ms, shown %d ms, jitter %d ms\n",
        i, l.TargetOn.Milliseconds(), shownMs, jitter)
}
```

`OnsetNS` and `OffsetNS` give the SDL3 nanosecond timestamps of the actual VSYNC flips that turned each stimulus on and off — the authoritative timing record, on the same clock as input events. `ActualOnset`/`ActualOffset` are Go-clock diagnostics on a different timebase; never subtract them from an event timestamp or use them as onsets. `OnsetNS` is zero only for a stimulus that was never displayed (zero on-frames); `OffsetNS` is the takedown flip, synthesised from the onset for the final element of a contiguous stream (which has no ISI flip of its own) so it stays on the SDL clock rather than reading a frame early.

Jitter below ±1 frame (±8–17 ms depending on monitor) is normal and expected. Larger jitter indicates system load or GPU driver issues.

Platform timing notes

`PresentStreamOfImages` provides the best timing accuracy that the operating system and display driver allow. The characteristics differ by platform:

Linux

- Without a compositing window manager (e.g. a plain X11 session with no compositor), SDL3's `Present()` blocks until the next VSYNC boundary. Onset jitter is typically **< 1 ms**.
- With a Wayland compositor or an X11 compositing WM (KWin, Mutter, Picom), the compositor controls buffer swaps. Onset jitter is typically **1–3 ms**; the compositor may add one frame (~17 ms at 60 Hz) of fixed latency.
- For the most reliable timing on Linux, disable the compositor or use a plain X11 session.

macOS (Metal)

- The macOS WindowServer compositor is **always active** — exclusive fullscreen does not bypass it. SDL3 submits each frame to the compositor, which forwards it to the display at the next VSYNC.
- `FlipTS` captures `sdl.TicksNS()` immediately after `Present()` returns; this reflects when the frame was *submitted to the compositor*, not when photons reached the

screen. The additional compositor latency is typically **1–2 frames** (~17-33 ms at 60 Hz).

Windows

- In **exclusive fullscreen** mode (`fullscreen=true`), the DWM (Desktop Window Manager) compositor is bypassed. Timing behaviour is similar to Linux without a compositor — jitter is typically **< 1 ms**.
- In **windowed mode**, DWM is always active and adds approximately one frame of compositor latency. Jitter is typically **1–3 ms**.
- For the most reliable timing on Windows, always run in fullscreen mode.

What `OnsetNS` measures

`TimingLog.OnsetNS` is recorded immediately after `screen.FlipTS()` returns, not at the moment photons reach the screen. On top of the compositor latency noted above, there is additional hardware pipeline latency (GPU scan-out, cable propagation, display panel response) of typically **0–2 frames**. This latency is constant across trials and does not affect within-experiment RT precision, but it must be accounted for if absolute (photodiode-verified) onset times are required.

Minimum duration

Because presentation is VSYNC-locked, durations shorter than one frame period (e.g. < 16.7 ms at 60 Hz) are rounded up to the nearest whole frame. A stimulus requested for 50 ms on a 60 Hz display is shown for exactly **3 frames (50.0 ms)**; one requested for 60 ms is shown for **4 frames (66.7 ms)**.

Audio stream

The same model applies to sounds:

```
// All tones must be pre-loaded before the stream
for _, t := range tones {
    t.PreloadDevice(exp.AudioDevice)
}

elements := stimuli.MakeRegularSoundStream(tones, 200*time.Millisecond, 100*time.Millis
events, logs, err := stimuli.PlayStreamOfSounds(elements)
```

Mixed stream

`PresentStreamOfStimuli` presents a single **sequential** stream that mixes visual and audio stimulus types. It reuses the VSYNC-locked visual loop, so visual elements are frame-accurate; audio elements are triggered once at the slot's first flip and the previous frame is **held** for the slot (the last stream visual is re-rendered every frame, so it stays rock-steady rather than relying on GPU buffer persistence) — place a visual just before a sound to keep it visible while the sound plays.

```
tone.PreloadDevice(exp.AudioDevice) // audio elements must be device-bound first
```

```
elements := []stimuli.StreamElement{
    {Stimulus: stimuli.NewTextLine("Ready?", 0, 0, control.White), DurationOn: 600 * ti
    {Stimulus: pic,  DurationOn: 800 * time.Millisecond}, // stays on screen during the
    {Stimulus: tone, DurationOn: 400 * time.Millisecond, DurationOff: 200 * time.Millis
}
events, logs, err := stimuli.PresentStreamOfStimuli(exp.Screen, elements, 0, 0)
```

Concurrent audio-visual overlap (a sound and an animating visual at the same instant) is not yet supported; the stream is strictly one slot at a time.

Per-stimulus onset triggers

To emit a hardware TTL — or run any action — aligned to each stimulus onset, use `PresentStreamOfStimuliHooks(screen, elements, x, y, onFrame, onOnset)`. The `onOnset OnsetCallback` (`func(index int, onsetNS uint64) error`) fires once per element **immediately after** its onset VSYNC flip, with `onsetNS == TimingLog[index].OnsetNS`. This is the *post-flip* counterpart to `FrameCallback`, which runs one frame earlier (pre-flip): a trigger emitted from `onOnset` coincides with the logged onset instead of leading it by a frame — the right hook for EEG/MEG synchronisation. Keep it short and non-blocking (emit an edge, clear it from a later `FrameCallback`; never sleep or call a blocking `Pulse`). The pure-audio equivalent is `PlayStreamOfSoundsHook(elements, onOnset)`, firing at the `Play()` instant. See the API reference and the `test_stream_trigger` example.

---

11. Audio

Sounds from files or embedded bytes

```
// From a file
snd := stimuli.NewSound("ping.wav")
snd.PreloadDevice(exp.AudioDevice)
snd.Play()
snd.Wait()   // block until playback finishes

// From embedded bytes (go:embed)
//go:embed assets/beep.wav
var beepWav []byte

snd := stimuli.NewSoundFromMemory(beepWav)
snd.PreloadDevice(exp.AudioDevice)
snd.Play()
```

Procedural tones

```
tone := stimuli.NewTone(1000.0, 200*time.Millisecond, 200) // 1 kHz, 200 ms, medium vol
tone.PreloadDevice(exp.AudioDevice)
tone.Play()
```

Volume is 0–255. Use `PreloadDevice` once; then call `Play` repeatedly.

Segment playback with fade

Play only part of a longer sound, with smooth fade-in and fade-out:

```
snd.PlaySegment(1.5, 3.0, 0.05) // play 1.5–3.0 s, 50 ms ramps
```

This is useful for stimuli like vowels extracted from longer recordings.

Built-in feedback sounds

```
exp.Audio.PlayBuzzer()   // play the embedded error/incorrect sound asynchronously
exp.Audio.PlayCorrect()  // play the embedded correct/reward sound asynchronously
```

Both return immediately; the sound plays in the background.

Synchronous vs asynchronous

`snd.Play()` starts playback and returns immediately. `snd.Wait()` blocks until it finishes. For trial timing where you need to know when a sound ended, call both:

```
snd.Play()
doSomethingElse()
snd.Wait()  // wait here if needed
```

For fire-and-forget feedback sounds, call `snd.Play()` without `snd.Wait()`.

Microphone recording

goxpyriment can capture audio from the system microphone via SDL3's recording API. The primary use cases are:

- **Picture naming** — measure the latency from image onset to the participant's first vocalization.
- **Vocal shadowing** — measure the latency between a spoken or tonal stimulus and the participant's repetition of it.

Open the microphone once, before the trial loop:

```
// nil uses the default recording spec: F32LE mono 44100 Hz
if err := exp.OpenMicrophone(nil); err != nil {
    log.Fatalf("microphone unavailable: %v", err)
}
// exp.Microphone is now ready; it is closed automatically by exp.End()
```

To select a custom format (e.g. higher sample rate or stereo):

```
spec := &sdl.AudioSpec{Format: sdl.AUDIO_F32LE, Channels: 1, Freq: 48000}
exp.OpenMicrophone(spec)
```

To list all connected recording devices and open a specific one:

```
devices, _ := apparatus.GetRecordingDevices()
for _, d := range devices {
    name, _ := d.Name()
    fmt.Println(d, name)
}
mic, _ := apparatus.NewMicrophoneFromDevice(devices[0], nil)
exp.Microphone = mic
```

Voice key — detecting vocal onset

A `VoiceKey` monitors the microphone and fires when the amplitude exceeds a threshold. It is the goxpyriment equivalent of a hardware voice key or PsychoPy's `OnsetVoiceKey`.

```
vk := apparatus.NewVoiceKey(exp.Microphone, 0.03, 128)
// threshold = 0.03 (RMS amplitude 0–1 for F32LE)
// windowSz  = 128 samples ≈ 2.9 ms at 44100 Hz
```

Per-trial pattern:

```
// 1. Arm: flush the mic buffer and start capturing.
//    Returns the SDL3 nanosecond timestamp of capture start.
captureStartNS, _ := vk.Arm()

// 2. Present the stimulus.
imageOnsetNS, _ := exp.ShowTS(picture)   // picture naming
// — or —
stimulusNS, _ := tone.PlaySyncedWithFlip(exp.Screen)  // shadowing

// 3. Wait for the vocal response (2-second window).
onsetNS, pcm, err := vk.WaitOnset(captureStartNS, 2000)
if errors.Is(err, apparatus.ErrVoiceTimeout) {
    // participant did not respond in time
}

// 4. Compute RT (both timestamps are on the SDL3 nanosecond clock).
rtMs := int64(onsetNS - imageOnsetNS) / 1_000_000
```

`WaitOnset` also returns `pcm` — the raw PCM bytes captured during the trial. Save them as WAV for offline verification (see below).

Threshold calibration. A value of 0.02–0.05 works in a quiet room. If you observe:

- **Spuriously short RTs (< 100 ms)** — the threshold is too low; it is triggering on breath noise, lip smacks, or microphone bleed from the speakers. Raise the

threshold or use headphones.
- **`detected = false` (missed responses)** — the threshold is too high for soft or breathy voices. Lower it.

Always verify using the saved WAV files rather than adjusting the threshold blindly.

Timing: microphone and screen on the same clock

Both `imageOnsetNS` (from `exp.ShowTS` or `screen.FlipTS`) and the voice onset computed by `WaitOnset` use `sdl.TicksNS()` — SDL3's monotonic nanosecond clock. No cross-clock conversion is needed:

```
vk.Arm()  ──────────────────────────────── captureStartNS  (sdl.TicksNS())
                │
                │  exp.ShowTS(picture)
                │  ↳ VSYNC flip ──────── imageOnsetNS   (sdl.TicksNS())
                │
                │  [participant starts speaking at sample N]
                │
                └── onsetNS = captureStartNS + N × 1e9 / sampleRate
                                                    (sdl.TicksNS() domain)

RT = (onsetNS − imageOnsetNS) / 1 000 000  [ms]
```

Call `vk.Arm()` just before showing the stimulus so the microphone is already capturing when the image appears. There is no need to arm it long in advance.

Saving WAV files for verification

Always save a WAV file for each trial and spot-check a random subset after the session. Voice keys can fire on breaths, lip smacks, or room noise; the WAV is the ground truth.

```
apparatus.WriteWAV(
    fmt.Sprintf("sub-%03d_trial-%02d.wav", exp.SubjectID, trial),
    exp.Microphone.Spec,
    pcm,
)
```

`WriteWAV` supports F32LE, S16LE, S32LE, and U8 formats and writes a standard RIFF/WAV file that any audio editor can open.

For post-hoc re-analysis of saved files (e.g. applying a different threshold), use `apparatus.ScanOnset`:

```
sampleIndex := apparatus.ScanOnset(pcm, newThreshold, 128)
if sampleIndex >= 0 {
    revisedOnsetNS = apparatus.SampleOnsetNS(captureStartNS, sampleIndex, int(exp.Micro
}
```

Acoustic feedback (shadowing and AV tasks)

When audio plays through laptop built-in speakers, the microphone picks up the playback signal and the voice key triggers on it rather than on the participant's voice. This produces spuriously short latencies (sometimes < 0 ms if the mic captures the speaker before the amp threshold is crossed).

Always use headphones for playback in any task that simultaneously records and plays audio.

If headphones are unavailable, a simple analysis-side fix is to reject any onset that falls within a guard interval after stimulus onset (e.g. discard `onsetNS - stimulusNS < 100 ms`).

---

12. Experimental Design and Randomization

When to use `design.Block` / `design.Trial`

The `design` package provides a structured `Experiment → Block → Trial` hierarchy with string-keyed factors. Use it when:

- You have a multi-block experiment and want `exp.Design.Blocks` to drive the loop.
- You want between-subjects Latin-square counterbalancing (`AddBWSFactor`).

For simple single-block experiments, a plain Go slice of a custom struct is often more readable (and type-safe). Both approaches are valid.

Building a design

```
block := design.NewBlock("main")
for _, cond := range []string{"congruent", "incongruent"} {
    t := design.NewTrial()
    t.SetFactor("condition", cond)
    t.SetFactor("soa", 200)
    block.AddTrial(t, 20, false)  // 20 copies, appended in order
}
block.ShuffleTrials()
exp.AddBlock(block, 1)
```

Iterate at runtime:

```
for _, blk := range exp.Design.Blocks {
    for _, trial := range blk.Trials {
        cond := trial.GetFactor("condition").(string)
        soa  := trial.GetFactor("soa").(int)
        // ...
    }
}
```

Randomization helpers

The `design` package provides randomization functions that work on any slice type:

```
// In-place shuffle (generic — works on any []T)
design.ShuffleList(mySlice)

// Random integer in [a, b] inclusive
design.RandInt(500, 1500)

// Random element from a slice
word := design.RandElement(wordList)

// Weighted coin flip
if design.CoinFlip(0.75) { /* 75% chance */ }

// Truncated normal sample in [a, b]
design.RandNorm(800.0, 1200.0)

// Shuffled integer range [first, last]
order := design.RandIntSequence(0, len(stimuli)-1)
```

Between-subjects counterbalancing

```
// Register once during setup
exp.AddBWSFactor("mapping", []interface{}{"F=left/J=right", "F=right/J=left"})

// At runtime — returns the condition assigned to this subject's ID
mapping := exp.GetPermutedBWSFactorCondition("mapping").(string)
```

The assignment follows a balanced Latin square so that conditions rotate across subjects (subject 1 → condition A, subject 2 → condition B, subject 3 → condition A, …).

Constrained shuffling

For designs where you need to prevent the same condition from appearing more than N times in a row, use block-level `AddTrial` with `randomPosition: true`, which inserts each trial at a random position rather than appending:

```
for _, cond := range conditions {
    t := design.NewTrial()
    t.SetFactor("condition", cond)
    block.AddTrial(t, 5, true)  // 5 copies each, randomly interleaved during construct
}
```

---

13. Embedding Stimuli and Trial Lists in the Executable

One of Go's most useful deployment features is `//go:embed`: a compiler directive that copies files from disk into the binary at build time. The result is a **single self-contained executable** that carries all its images, sounds, and trial-list CSV files with it — no separate `assets/` folder, no installer, no missing-file errors on lab computers.

> **When to embed.** Embedding is the right choice for stimuli that are fixed at design time: images, sounds, and pre-generated trial lists. It is not appropriate for stimuli you generate procedurally at runtime (textures drawn with `Canvas`, tones created with `NewTone`, text rendered from `NewTextLine`), or for data files that are written during the experiment.

13.1 Project layout

Collect all static assets in a subdirectory alongside `main.go`:

```
myexperiment/
  main.go
  assets/
    stim_A.png
    stim_B.png
    feedback_correct.wav
    trials.csv
```

13.2 Embedding individual image and sound files

Import `_ "embed"` (blank identifier, needed to activate the directive) and declare a `[]byte` variable per file:

```go
package main

import _ "embed"

//go:embed assets/stim_A.png
var stimAImg []byte

//go:embed assets/stim_B.png
var stimBImg []byte

//go:embed assets/feedback_correct.wav
var feedbackWav []byte
```

Pass the byte slices to the `FromMemory` constructors:

```go
import "github.com/chrplr/goxpyriment/stimuli"

stimA := stimuli.NewPictureFromMemory(stimAImg, 0, 0)
stimB := stimuli.NewPictureFromMemory(stimBImg, 0, 0)
feedback := stimuli.NewSoundFromMemory(feedbackWav)
```

These constructors behave identically to `NewPicture` and `NewSound`; textures are still allocated lazily on first `Draw`, and `PreloadAllVisual` / `PreloadDevice` work the same way.

13.3 Embedding a whole stimulus directory

When you have many images (e.g. one per trial) it is impractical to declare a variable for each one. Embed the whole directory as an `embed.FS` and iterate over it at runtime:

```
package main

import "embed"

//go:embed assets/stimuli
var stimuliFS embed.FS
```

Load every PNG in the directory into a map:

```
import (
    "fmt"
    "github.com/chrplr/goxpyriment/stimuli"
)

pics := map[string]*stimuli.Picture{}

entries, err := stimuliFS.ReadDir("assets/stimuli")
if err != nil {
    log.Fatal(err)
}
for _, e := range entries {
    if e.IsDir() {
        continue
    }
    data, err := stimuliFS.ReadFile("assets/stimuli/" + e.Name())
    if err != nil {
        log.Fatal(err)
    }
    pics[e.Name()] = stimuli.NewPictureFromMemory(data, 0, 0)
}

// Use by filename:
exp.Show(pics["face_happy.png"])
```

Or load a specific file by name:

```
data, _ := stimuliFS.ReadFile(fmt.Sprintf("assets/stimuli/face_%s.png", condition))
stim := stimuli.NewPictureFromMemory(data, 0, 0)
```

13.4 Embedding a CSV trial list

A CSV file that defines trial order and parameters is a natural fit for embedding. The workflow is: embed → read into `[]byte` → wrap in `bytes.NewReader` → parse with `encoding/csv`.

Example `assets/trials.csv`:

```
condition,soa_ms,correct_key
congruent,200,F
incongruent,200,J
congruent,400,F
incongruent,400,J
```

Embedding and parsing:

```
package main

import (
    "bytes"
    _ "embed"
    "encoding/csv"
    "log"
    "strconv"
)

//go:embed assets/trials.csv
var trialsCSV []byte

type trial struct {
    condition  string
    soaMs      int
    correctKey string
}

func loadTrials() ([]trial, error) {
    r := csv.NewReader(bytes.NewReader(trialsCSV))
    records, err := r.ReadAll()
    if err != nil {
        return nil, err
    }
    var trials []trial
    for i, rec := range records {
        if i == 0 {
            continue // skip header row
        }
        soa, _ := strconv.Atoi(rec[1])
        trials = append(trials, trial{
            condition:  rec[0],
            soaMs:      soa,
```

```
            correctKey: rec[2],
        })
    }
    return trials, nil
}
```

For **tab-separated** files, set `r.Comma = '\t'` immediately after creating the reader.

For **semicolon-separated** files (common in European locales), set `r.Comma = ';'`.

Use the loaded trials in the experiment loop:

```
trials, err := loadTrials()
if err != nil {
    log.Fatalf("loading trials: %v", err)
}

err = exp.Run(func() error {
    exp.ShowInstructions("Press F or J to respond.")
    for _, t := range trials {
        exp.Blank(500)
        stim := stimuli.NewTextLine(t.condition, 0, 0, control.White)
        key, rt, _ := exp.ShowAndGetRT(stim, []control.Keycode{control.K_F, control.K_J
        correct := string(key.KeyName()) == t.correctKey
        exp.Data.Add(t.condition, t.soaMs, rt, correct)
    }
    return control.EndLoop
})
```

13.5 Mixing embedded and runtime-generated stimuli

Embedding is selective — embed only what is fixed, generate the rest at runtime:

```
//go:embed assets/mask.png
var maskImg []byte

// Fixed stimulus: loaded from embedded bytes
mask := stimuli.NewPictureFromMemory(maskImg, 0, 0)

// Dynamic stimulus: generated from trial parameters
target := stimuli.NewTextLine(word, 0, 0, control.White)
```

13.6 Build and deployment

Nothing changes in the build command:

```
go build -o myexperiment .
```

The `assets/` directory must exist on disk at build time but is **not needed at runtime**. Copy the single binary to any lab computer running the same OS:

```
# Cross-compile for Windows from Linux
GOOS=windows GOARCH=amd64 go build -o myexperiment.exe .

# Cross-compile for macOS (Apple Silicon)
GOOS=darwin GOARCH=arm64 go build -o myexperiment-mac .
```

The embedded assets are included in all cross-compiled outputs automatically.

> **Binary size.** Each embedded file adds its uncompressed size to the binary. A typical experiment with a few dozen PNG images and WAV files will add 5–50 MB — acceptable for a self-contained research tool. For very large stimulus sets (hundreds of high-resolution images, video files), consider distributing them as a separate archive alongside the binary instead of embedding.

---

14. Animated Stimuli

Three functions run self-contained VSYNC-locked animation loops. All three:

- Disable GC for the duration of the loop.
- Drain the SDL event queue before the first frame.
- Return a `MotionResult` with the response key, mouse button, and reaction time.

```
type MotionResult struct {
    Key    sdl.Keycode  // interrupt key pressed (0 if none)
    Button uint8        // mouse button (0 if none)
    RTms   int64        // ms from first frame to response; total elapsed on timeout
}
```

Moving dot cloud

```
result, err := stimuli.PresentMovingDotCloud(
    exp.Screen,
    100,                          // number of dots
    3.0,                          // dot radius (px)
    150.0,                        // cloud radius (px)
    control.Origin(),             // center at (0, 0)
    150.0,                        // speed in px/sec
    5000,                         // max duration ms (0 = infinite)
    []control.Keycode{control.K_SPACE}, // interrupt keys
    false,                        // catch mouse?
    control.White,                // dot color
    control.Color{A: 0},          // background (transparent)
)
fmt.Printf("RT: %d ms\n", result.RTms)
```

Drifting grating

```
result, err := stimuli.PresentMovingGrating(
    exp.Screen,
    400, 400,              // width, height (px)
    control.Origin(),      // center
    0.0,                   // orientation (degrees; 0 = vertical bars drifting right)
    0.05,                  // spatial frequency (cycles per pixel)
    2.0,                   // temporal frequency (Hz)
    0.8,                   // contrast [0, 1]
    0.5,                   // mean luminance [0, 1]
    3000,                  // max duration ms
    []control.Keycode{control.K_SPACE},
    false,
)
```

Drifting Gabor patch

```
result, err := stimuli.PresentMovingGabor(
    exp.Screen,
    200,                   // bounding box size (px); at least 6×sigma
    30.0,                  // sigma (Gaussian SD in px)
    control.Origin(),      // center
    45.0,                  // orientation
    0.05,                  // spatial frequency (cycles/px)
    3.0,                   // temporal frequency (Hz)
    0.9,                   // contrast
    0.5,                   // mean luminance
    0,                     // max duration (0 = until response)
    []control.Keycode{control.K_SPACE},
    false,
)
```

---

15. Video

goxpyriment plays video in two formats, and the choice is a deliberate trade between **timing determinism** and **file size / convenience**. Neither is a general-purpose media player: both exist to put moving images on screen at a known time.

	`.gv`	MPEG-1 (`.mpg`)
Type	`stimuli.GvVideo`, `stimuli.PlayGv`, `media.Movie`	`stimuli.Video`
Frame cadence	VSYNC-locked, GC disabled	wall-clock, drops frames under load

	`.gv`	MPEG-1 (`.mpg`)
Decode cost	constant per frame	varies with I/P/B frame type
Audio	none	MP2 soundtrack
Seeking	O(1), every frame independent	sequential only
Size (10 s, 960×400)	~41 MB	~2 MB
Use for	measured stimuli	instructions, fillers, long clips

Rule of thumb: if the onset time of a frame ends up in your data file, use `.gv`. If the participant is just watching something between trials, MPEG-1 is far more convenient.

Start here

Almost everyone arrives with an `.mp4` and one of two goals.

"I want to play a movie, with its sound." You need MPEG-1, because `.gv` has no audio track. Convert once, then play:

```
ffmpeg -i movie.mp4 -c:v mpeg1video -b:v 1500k \
                    -c:a mp2 -b:a 192k -ar 44100 -ac 2 \
                    -f mpeg movie.mpg
```

```
v, _ := stimuli.NewVideo(exp.Screen, "movie.mpg")
defer v.Close()
v.PreloadDevice(exp.AudioDevice)   // without this it plays silently
v.Play()
```

Full walkthrough in 15.2. Frame onsets are approximate — fine for instructions and filler, not for measured stimuli.

"I need frame-accurate timing, and maybe a TTL trigger." You need `.gv`. Convert once, then play:

```
make gv-convert
_build/gv-convert movie.mp4 movie.gv    # needs ffmpeg for non-MPEG-1 input
```

```
events, logs, err := stimuli.PlayGv(exp.Screen, "movie.gv", 0, 0)
```

`logs` tells you afterwards whether any frame was late. Full walkthrough, including how to fire a trigger on a chosen frame, in 15.1. If you also need sound, play a separate `stimuli.Sound` alongside it — which additionally gives you independent control of audio onset (see Section 11).

15.1 `.gv` files — the timing-critical format

`.gv` is the *Extreme GPU Friendly Video Format*, originally from ofxExtremeGpuVideo. Each frame is an independent, LZ4-compressed block of raw RGBA, followed by an index table of `(address, size)` pairs at the end of the file.

That layout is what buys the timing guarantee. There is no inter-frame prediction, so presenting frame *n* costs a seek, one LZ4 decompression, and a texture upload — the same work for every frame, whatever the content. Compare this with H.264, where an I-frame costs several times a B-frame and the resulting jitter is *periodic*, aligned to the GOP boundary. That is the pathological case for frame-accurate onsets, and it is exactly what `.gv` avoids.

You pay for it in disk space. A frame is `width × height × 4` bytes before compression; LZ4 is fast rather than thorough, so expect files roughly an order of magnitude larger than an equivalent MPEG-1. The 10 s 960×400 clip in the table above is 366 MB raw, 41 MB as `.gv`, and 2 MB as MPEG-1.

`.gv` carries **no audio track**. Play a separate `stimuli.Sound` alongside it, which also gives you independent control over audio onset (see Section 11).

Creating a `.gv` file `cmd/gv-convert` converts a video directly:

```
make gv-convert                          # builds _build/gv-convert

gv-convert clip.mpg clip.gv              # MPEG-1 input: pure Go, no ffmpeg
gv-convert movie.mp4 movie.gv            # anything else: needs ffmpeg on PATH
gv-convert frames/ anim.gv               # a directory of numbered images
gv-convert -fps 30 movie.mov movie.gv    # resample to 30 fps
gv-convert -max 100 long.mp4 preview.gv  # first 100 frames only
```

MPEG-1 program streams are decoded in pure Go, so that path needs nothing installed. Every other video format is piped through `ffmpeg`, which must be on `PATH`; if it is missing, the tool says so and names both decoders it tried.

From an image sequence Passing a **directory** converts the `.png`/`.jpg`/`.jpeg`/`.gif` files it contains. This is the right route when frames are generated programmatically (drifting gratings, custom animations) rather than filmed. Two behaviours differ from a naive image-sequence converter, both deliberately:

- **Frames are ordered numerically, not lexicographically.** With unpadded names a plain string sort puts `frame_10.png` before `frame_2.png`, silently playing the sequence out of order. `gv-convert` sorts by numeric value and prints a note when the two orders disagree, so you can see that it mattered.
- **All frames must be the same size.** A mismatch is an error, not a silent crop: a wrongly-sized stimulus is the kind of fault that surfaces only at analysis. Pass `-force-size` to crop/pad against the first frame instead.

A directory has no inherent frame rate, so `-fps` sets it (default 30).

The standalone images2gv does the same job without requiring the goxpyriment tree, and targets other `.gv` players such as `ofxExtremeGpuVideo` and ebiten.

Inspecting a `.gv` file `cmd/gv-getinfo` prints what a `.gv` file actually contains — useful to confirm a conversion produced the frame rate and dimensions you expected, and to check that a file will play at all:

```
make gv-getinfo                  # builds _build/gv-getinfo

$ gv-getinfo stim.gv
file        : stim.gv
frame size  : 960 x 400 px  (1.5 MB raw per frame)
frame count : 250
frame rate  : 25.000 fps   (duration 10.00 s)
pixel format: raw RGBA
file size   : 40.8 MB
frame data  : 40.8 MB compressed from 366.2 MB  (9.0x, 11.2% of raw)
frame sizes : min 5.9 KB, median 104.3 KB, max 611.2 KB
```

`-frames` lists every frame's offset and compressed size; `-q` prints one tab-separated line per file for scripting. The tool also cross-checks the header against the index table and warns about inconsistencies — a `.gv` whose header disagrees with its index loads but plays incorrectly, which is otherwise hard to diagnose from a black screen.

Files written by other tools may use DXT1/DXT3/DXT5 or BC7 block compression instead of raw RGBA. `gv-getinfo` flags these: goxpyriment's reader LZ4-decompresses straight into a texture and never DXT-decodes, so only `raw RGBA` is playable.

Playing a `.gv` file One-shot, for when the video *is* the trial:

```
events, logs, err := stimuli.PlayGv(exp.Screen, "stim.gv", 0, 0)
```

`PlayGv` disables GC, locks to VSYNC, and returns a per-frame `TimingLog` you can write to your data file to verify that no frame was late. It exits on ESC.

The clip's frame rate must divide the refresh rate `PlayGv` plays at the rate stored in the `.gv` header, holding each video frame for however many refreshes that takes — a 30 fps clip on a 60 Hz display is shown for 2 refreshes per frame. Both rates are rounded first, so a display reporting 59.94 Hz and a clip authored at 29.97 fps behave as 60 and 30.

Only exact integer ratios are supported. A 24 fps clip on a 60 Hz display would need 2.5 refreshes per frame, so `PlayGv` refuses it rather than play it at the wrong speed or with uneven onsets:

```
display refresh 60 Hz is not an integer multiple of the video's 24 fps
(2.500 frames per refresh); re-encode the clip at a divisor of 60 fps,
e.g. gv-convert -fps 20
```

The 3:2 pulldown that would otherwise be required makes frame onsets uneven, which defeats the reason for using `.gv` at all. **Choose the clip's frame rate to divide the refresh rate of the machine it will run on** — at 60 Hz that means 60, 30, 20, 15, 12, 10 …; `gv-convert -fps` sets it, and `gv-getinfo` reports what a file actually claims.

Interactive, when you need to draw around the video or respond mid-clip:

```
v, err := stimuli.NewGvVideo("stim.gv")
if err != nil {
    log.Fatal(err)
}
defer v.Close()

v.Play()
exp.Run(func() error {
    if err := v.Update(exp.Screen); err == io.EOF {
        return control.EndLoop
    }
    exp.Screen.Clear()
    v.DrawAt(exp.Screen, &control.FRect{X: 100, Y: 50, W: 640, H: 480})
    return exp.Screen.Flip()   // presents and holds one frame: see Section 5
})
```

`Flip` is an alias for `Update`; both hold one frame. On drivers that present without blocking (triple/mailbox buffering), `Update` returns immediately and the loop runs faster than the display, swallowing frames.

`Width`, `Height`, `FrameCount` and `FPS` are read from the header at construction; the GPU texture is allocated lazily on the first `Update` or `Draw`. For playing several clips back to back without a gap, use `media.MovieManager` (see `docs/MediaMovies.md`).

Firing a trigger on a chosen frame `PlayGvFunc` is `PlayGv` with a per-frame callback, invoked immediately after the flip that puts each frame on screen:

```
events, logs, err := stimuli.PlayGvFunc(exp.Screen, "stim.gv", 0, 0,
    func(ctx stimuli.GvFrameContext) error {
        if ctx.Frame == onsetFrame {
            trig.SetHigh(0)              // rising edge tied to this flip
        }
        if ctx.Frame == onsetFrame+3 {   // ~50 ms later at 60 Hz
            trig.SetLow(0)
        }
        return nil                       // control.EndLoop to stop early
    })
```

`ctx.OnsetNS` is the SDL timestamp of that frame's **first** flip, in the same reference frame as event timestamps. `ctx.Frame` is the 0-based video frame index and `ctx.Hold` the number of refreshes it is shown for. The callback runs inside the GC-disabled VSYNC loop, so it must not allocate heavily or sleep — see the warning below.

`tests/test_gv_sync` is a complete worked example: it plays a generated flash train and fires a TTL pulse at each bright onset for photodiode verification.

If you need to draw around the video as well, drive the loop yourself instead. `CurrentFrame()` returns the frame now in the GPU texture — the one the next flip will show — so raise the line immediately *after* the flip that displays it:

```
const onsetFrame = 120        // the frame whose onset you want marked
const pulseFrames = 3         // ~50 ms at 60 Hz

// Falls back to a no-op device when no hardware is present, so this code
// still runs on a machine without a trigger box.
trig, _, err := triggers.AutoDetectDLPIO8()
if err != nil {
    log.Fatal(err)
}
defer trig.Close()

lowAt := -1
v.Play()
exp.Run(func() error {
    if err := v.Update(exp.Screen); err == io.EOF {
        return control.EndLoop
    }
    exp.Screen.Clear()
    v.Draw(exp.Screen)

    flipNS, err := exp.Screen.FlipTS()
    if err != nil {
        return err
    }
    if v.CurrentFrame() == onsetFrame {
        trig.SetHigh(0)                        // rising edge, right after the flip
        lowAt = v.CurrentFrame() + pulseFrames
        exp.Data.Add(trialNo, onsetFrame, flipNS)   // trialNo: your own counter
    }
    if v.CurrentFrame() == lowAt {
        trig.SetLow(0)
    }
    return nil
})
```

Do not call `trig.Pulse` here. `Pulse` is `SetHigh`, `time.Sleep`, `SetLow` — it blocks for the whole pulse width, which inside a VSYNC-locked loop costs you frames. Splitting it across iterations as above sleeps not at all, and ties the rising edge (the thing your amplifier timestamps) to a known flip. See Section 17 for why wrapping `Pulse` in a goroutine is not the fix.

`flipNS` shares its reference frame with SDL event timestamps, so it subtracts directly from a keypress timestamp to give a hardware-precision RT relative to the video frame (see Section 6).

Verifying that no frame was late `PlayGv` returns a `[]VideoFrameLog`, one entry per frame:

```
type VideoFrameLog struct {
    Frame         int    // 0-based video frame index
    FlipNS        uint64 // SDL nanosecond timestamp after the flip
    SkippedFrames int    // display frames late (0 = on time)
}
```

Scan `SkippedFrames` after a trial and record the worst value, or abort the session if it exceeds what your paradigm tolerates. A clip that drops frames on the experiment machine is a data-quality problem you want to detect during piloting, not discover in the analysis.

15.2 MPEG-1 files — the convenient format

`stimuli.Video` decodes MPEG-1 in pure Go via gen2brain/mpeg. No ffmpeg, no cgo, no conversion step — you point it at a `.mpg` and it plays, with sound.

The cost is that playback is driven by the wall clock, not by VSYNC. `Update` computes the target frame from elapsed time and *drops* frames to catch up if decoding falls behind. Playback stays the right length, but the onset of any particular frame is only approximate. **Do not put a frame onset from `stimuli.Video` in a data file** — use `.gv` for that.

Converting an mp4 (or anything else) to MPEG-1 `stimuli.Video` plays only MPEG-1, so anything else has to be converted once, up front. This is the command:

```
ffmpeg -i input.mp4 -c:v mpeg1video -b:v 1500k \
                    -c:a mp2 -b:a 192k -ar 44100 -ac 2 \
                    -f mpeg output.mpg
```

Reading it piece by piece:

Part	Why
`-c:v mpeg1video`	MPEG-**1** video. MPEG-2 in a `.mpg` will not play
`-b:v 1500k`	video bitrate; raise for large frames, lower for smaller files
`-c:a mp2`	MP2 is the only audio codec an MPEG-1 program stream carries
`-ar 44100 -ac 2`	44.1 kHz stereo; safe defaults
`-f mpeg`	**the load-bearing flag** — selects the MPEG-1 system muxer

`-f mpeg` is the one people get wrong. `-f vob` looks equivalent and produces an MPEG-2 program stream that is rejected at load. If you omit `-f` entirely, ffmpeg guesses from the `.mpg` extension and usually gets it right, but being explicit costs nothing.

Drop the three audio flags for a silent clip. To check the result before wiring it into an experiment, see the next section.

Format requirements The decoder is strict, and the two most common failures are worth knowing:

- It must be an **MPEG-1 program stream** — the file starts with the pack header `00 00 01 BA`. A raw elementary stream (starting `00 00 01 B3`) is rejected with `invalid MPEG-PS`. An elementary stream also cannot carry audio at all, whatever the filename suggests.
- The video must be **MPEG-1**, not MPEG-2. A `.mpg` containing MPEG-2 is out of scope for the library even though the extension is the same.

Check a file before blaming your code:

```
file clip.mpg     # want: "MPEG sequence, v1, system multiplex"
ffprobe clip.mpg  # want: format_name=mpeg, codec_name=mpeg1video (+ mp2 audio)
```

If `file` says *"MPEG sequence, v2"* the video is MPEG-2; if it does not say *"system multiplex"* you have an elementary stream. Either way, re-run the conversion above. A file with only one stream listed by `ffprobe` has no soundtrack, whatever its name promises.

```
v, err := stimuli.NewVideo(exp.Screen, "clip.mpg")
if err != nil {
    log.Fatal(err)
}
defer v.Close()

// Enable the MP2 soundtrack. Safe to call unconditionally: it is a no-op
// returning nil when the file has no audio stream.
if err := v.PreloadDevice(exp.AudioDevice); err != nil {
    log.Printf("audio unavailable: %v", err)
}

v.Play()
exp.Run(func() error {
    if err := v.Update(); err == io.EOF {   // note: no screen argument
        return control.EndLoop
    }
    exp.Screen.Clear()
    v.Draw(exp.Screen, 0, 0)
    exp.Screen.Update()
    return nil
})
```

Playing an MPEG-1 file Audio is off until `PreloadDevice` is called. Without it the video plays silently, and audio packets are discarded at the demuxer — leaving decoding enabled with nothing draining the buffer would grow it for the length of the clip. Check `v.HasAudio` if you need to know whether a soundtrack exists, and use `v.SetVolume(gain)` to scale it.

`Update` keeps 100 ms of decoded audio queued on the SDL device. Audio is free-running once queued, so it stays aligned with the video to within that depth. Two consequences: `Pause` clears the queue so sound stops with the image (resuming leaves audio up to 100 ms ahead), and `Rewind` clears it too so a restarted clip does not play over its own tail.

---

16. Gamma Correction and Luminance Linearity

The gamma problem

Standard monitors apply a power-law transfer function when converting digital RGB values to physical luminance:

$L(V) = k \cdot (V/255)^{\gamma}$ $\gamma \approx 2.2$ for sRGB/LCD

This means equal steps in RGB values do **not** produce equal steps in physical luminance (cd/m²). For example, with $\gamma = 2.2$:

RGB value	Physical luminance (% of max)
0	0%
64	4.9%
128	21.6%
192	52.2%
255	100%

This matters in psychophysics experiments where luminance values are specified as physical quantities (e.g. contrast masks, threshold estimation, magnitude estimation).

Inverse-gamma LUT

goxpyriment corrects for this by pre-computing a 256-entry look-up table (LUT) per channel. For each desired linear luminance value v (0–255), the LUT stores the digital value required to achieve it:

$LUT[v] = 255 \cdot (v/255)^{(1/\gamma)}$

Applying the LUT before sending a color to the renderer means the monitor's forward gamma converts it back to the intended linear value.

Enabling gamma correction

```
// After exp.Initialize() and before the trial loop:
exp.SetGamma(2.2)   // typical sRGB monitor

// Or with per-channel calibration (from photometer measurements):
exp.GammaCorrector = apparatus.NewGammaCorrector(2.1, 2.2, 2.3)
```

Then wrap every color with `exp.CorrectColor`:

```
// Specify colors in linear luminance space (0–255).
// exp.CorrectColor maps them to the physical digital values.
disk := stimuli.NewFilledCircle(exp.CorrectColor(control.RGB(128, 128, 128)), radius)
```

When `SetGamma` has not been called, `CorrectColor` is a no-op that returns the color unchanged, so it is safe to always call it.

Measuring your monitor's gamma

The most accurate approach is photometer-based: measure luminance at several RGB levels, fit the model L(V) = k·(V/255)^γ, and pass the resulting γ to `SetGamma`. Without a photometer, γ = 2.2 is a reasonable default for modern sRGB displays.

Practical example

The `examples/Magnitude-Estimation-Luminosity` example accepts a `-gamma` flag:

```
go run main.go -gamma 2.2
```

With the flag set, the 7 luminance levels (10, 25, 50, 100, 150, 200, 255) are treated as linear targets and corrected before display, so they produce physically uniform luminance steps.

---

17. Hardware Triggers and TTL Devices

EEG and MEG recordings require a synchronisation signal — a short TTL pulse sent at the exact moment a stimulus is presented — so that electrophysiological data can be time-locked to experimental events. The `triggers` package provides this, along with the ability to read TTL inputs from response hardware such as fiber-optic response pads.

Concepts

All trigger devices in goxpyriment share two interfaces:

- **`OutputTTLDevice`** — send trigger codes (set lines HIGH/LOW, generate pulses)
- **`InputTTLDevice`** — read response button states (poll or block)

Lines are **0-indexed (0–7)**. Bit N of a bitmask drives line N.

Sending triggers (EEG event codes)

```
import (
    "github.com/chrplr/goxpyriment/triggers"
    "time"
)
```

```
// Auto-detect a DLP-IO8-G; falls back to NullOutputTTLDevice (no-op) if absent.
out, _, err := triggers.AutoDetectDLPIO8()
if err != nil { log.Fatal(err) }
defer out.Close()

// In the trial loop:
onset, _ := exp.ShowTS(stim)
out.Pulse(0, 10*time.Millisecond)  // 10 ms pulse on line 0 = EEG event marker
```

For the NeuroSpin MEGTTLBox:

```
box, err := triggers.NewMEGTTLBox("/dev/ttyACM0")
defer box.Close()

box.Pulse(0, 5*time.Millisecond)   // single line
box.PulseMask(0b00000011, 5*time.Millisecond)  // lines 0 and 1 simultaneously
```

Timing advice

The output pulse should be sent as close as possible to `exp.ShowTS(stim)`. Because both `Screen.FlipTS` and the trigger output use the system's real-time clock, the latency between the VSYNC flip and the TTL edge is typically under 1 ms. The EEG amplifier records this edge, and the exact onset can be recovered by subtracting `int64(triggerEdgeNS - onsetNS)` if the amplifier's sample clock is synchronised.

`Pulse` blocks — and a goroutine is not the fix

`Pulse(line, d)` is `SetHigh`, `time.Sleep(d)`, `SetLow`. It blocks for the full width. That is fine in an ordinary trial loop, where the sleep overlaps a stimulus you were going to display anyway — but inside a per-frame loop (`.gv` playback, `PresentStreamOfStimuliFunc`, any VSYNC-locked animation) it costs you frames.

The tempting fix is `go trig.Pulse(0, 5*time.Millisecond)`. Don't:

- **It races.** `ParallelPort.SetHigh` is an unsynchronised read-modify-write on a shadow register, so overlapping calls can lose an update and leave a line stuck HIGH or send a wrong code. `LabJackT4` shares one Modbus TCP connection; the serial devices give no ordering guarantee across goroutines. Only `FT232HTrigger` and `LinuxGPIOTrigger` hold a mutex. These failures are silent and intermittent, and they corrupt the event record in your EEG file.
- **It moves the edge onto the Go scheduler.** The amplifier timestamps the *rising* edge — that is your event marker. Blocking `Pulse` raises the line immediately and only the *width* costs time; `go Pulse` defers the `SetHigh` to whenever the scheduler runs it, adding jitter to the one edge that must not have any.
- **It outlives its device.** If the trial ends and the device closes while the goroutine sleeps, `SetLow` fires on a closed port.

In a per-frame loop, split the pulse across iterations instead — no sleeping at all, and the rising edge is locked to a known flip:

```
if thisIsTheOnsetFrame {
    trig.SetHigh(0)              // rising edge, right after the flip
    lowAt = frameIndex + 3       // ~50 ms at 60 Hz
}
if frameIndex == lowAt {
    trig.SetLow(0)
}
```

The width quantises to whole frames, which is well above the ~1 ms most amplifiers need. Section 15 shows this applied to `.gv` playback.

Reading response inputs (FORP pads)

The MEGTTLBox wires a fiber-optic response pad (fORP) to its 8 TTL input lines. Use `WaitForInput` directly or via the `ResponseDevice` wrapper (section 7):

```
// Via InputTTLDevice directly
_ = box.DrainInputs(ctx)                    // clear stale presses
mask, rt, err := box.WaitForInput(ctx)
buttons := triggers.DecodeMask(mask)
for _, b := range buttons {
    fmt.Println(b)  // e.g. "left blue", "right red"
}

// Via ResponseDevice
rd := apparatus.NewTTLResponseDevice(box, 5*time.Millisecond)
_ = rd.DrainResponses(ctx)
resp, _ := rd.WaitResponse(ctx)
// resp.Code == bitmask, resp.Precise == false (software poll)
```

fORP button mapping (NeuroSpin wiring):

FORPButton constant	Line	Arduino pin	STI channel
`FORPLeftBlue`	0	D22	STI007
`FORPLeftYellow`	1	D23	STI008
`FORPLeftGreen`	2	D24	STI009
`FORPLeftRed`	3	D25	STI010
`FORPRightBlue`	4	D26	STI012
`FORPRightYellow`	5	D27	STI013
`FORPRightGreen`	6	D28	STI014
`FORPRightRed`	7	D29	STI015

FT232H (Adafruit FT232H breakout, Linux)

A USB-to-MPSSE bridge. AD0–AD7 are TTL output lines; AC0–AC7 are TTL input lines. No libftdi or D2XX installation needed — the driver uses Linux usbfs directly.

```
box, err := triggers.NewFT232H()
defer box.Close()

box.Pulse(0, 5*time.Millisecond)
mask, rt, _ := box.WaitForInput(ctx)

// Auto-detect (returns NullOutputTTLDevice if no device found):
out, _ := triggers.AutoDetectFT232H()
```

Prerequisites: unload the `ftdi_sio` kernel module (`sudo rmmod ftdi_sio`) and ensure the user has rw access to the USB device node (udev rule or `plugdev` group membership).

GPIO (Raspberry Pi and other Linux SBCs)

Drives any 8 GPIO pins as TTL outputs and reads any other 8 pins as TTL inputs, using the Linux GPIO character device v2 API. Works on Raspberry Pi (BCM pin numbers), Rock Pi, BeagleBone, Jetson, and any Linux SBC with kernel $\geq$ 5.10.

```
box, err := triggers.NewLinuxGPIOTrigger(
    triggers.WithGPIOOutputPins([8]int{17, 27, 22, 5, 6, 13, 19, 26}),
    triggers.WithGPIOInputPins([8]int{12, 16, 20, 21, 4, 25, 24, 23}),
)
defer box.Close()

box.Pulse(0, 5*time.Millisecond)
mask, rt, _ := box.WaitForInput(ctx)
```

Output-only and input-only configurations are both valid. Prerequisites: kernel $\geq$ 5.10; user in the `gpio` group.

LabJack T4 (Modbus TCP)

Communicates over Ethernet via Modbus TCP — no LabJack SDK or driver required. The eight output lines are DIO4–DIO11 (FIO4–FIO7 + EIO0–EIO3) and the eight input lines are DIO12–DIO19 (EIO4–EIO7 + CIO0–CIO3); `WithT4OutputBase` / `WithT4InputBase` move those groups. DIO0–DIO3 are the T4's dedicated analog inputs and are unavailable as digital lines.

```
box, err := triggers.NewLabJackT4("192.168.1.100")
defer box.Close()

box.Pulse(0, 5*time.Millisecond)
mask, rt, _ := box.WaitForInput(ctx)
```

The device must be reachable on the local network. Default Modbus port 502 is used; append `:port` to the host string to override.

DLP-IO20 (USB-CDC serial)

A 20-channel USB module, 5 V logic. It has more digital channels (17) than the 8-bit TTL interfaces can express, so interface lines 0–7 address a *window* of 8 channels — `AN0`–`AN7` for output and `AN8`–`AN13` + `RB6`/`RB7` for input by default. Every channel remains reachable through the channel-level methods.

```
d, err := triggers.NewDLPIO20("/dev/ttyUSB0")   // or triggers.AutoDetectDLPIO20()
defer d.Close()

d.Send(0b00000101)                    // lines 0 and 2 → AN0, AN2
d.SetChannelHigh(triggers.IO20_RA4)   // a channel outside the window
v, _ := d.ReadChannel(triggers.IO20_AN12)
```

Remap the windows with `WithIO20OutputChannels` / `WithIO20InputChannels` (8 channels each, no duplicates, no overlap between the two groups).

Two caveats. The device has no write-all command, so `Send` issues one packet per line and the 8 lines change over ~3.5 ms rather than together — use `SetHigh` on a single line when trigger onset matters. And the driver is **untested on real hardware**: it was written from the datasheet, so verify it with `tests/test_dlpio20` before an experiment depends on it.

Supported devices

Device	Type	Output	Input	Connection	Notes
DLP-IO8-G	`DLPIO8`	✓	✓	USB-CDC serial	ASCII protocol
DLP-IO20	`DLPIO20`	✓	✓	USB-CDC serial	Packet protocol; 17 digital channels via 8-line windows; 5 V; untested on hardware
NeuroSpin MEGTTL-Box	`MEGTTLBox`	✓	✓	USB serial	Arduino Mega; binary protocol; fORP input
Adafruit FT232H	`FT232HTrigger`	✓	✓	USB (usbfs)	Linux only; no libftdi needed

Device	Type	Output	Input	Connection	Notes
Linux GPIO	`LinuxGPIOTrigger`	✓	✓	GPIO pins	Any Linux SBC; kernel ≥ 5.10
LabJack T4	`LabJackT4`	✓	✓	Ethernet (Modbus TCP)	No SDK needed
LPT parallel port	`ParallelPort`	✓	—	LPT	Linux only; ppdev ioctl
None / fallback	`NullOutputTTLDevice`	✓ (no-op)	—	—	Returned by `AutoDetectDLPI08`, `AutoDetectFT232H`

Networked recording control (non-TTL)

The `triggers` package also hosts two devices that are **not** TTL bitmask hardware: they mark events in — and control — an *external recorder* over TCP/IP. They share the package because their purpose is the same (annotate a recording so it can be aligned with your paradigm), but they implement neither `OutputTTLDevice` nor `InputTTLDevice`; each has its own richer API. `SerialPort` (a generic UART wrapper) is the third non-TTL member.

NetStation (EGI EEG event markers over TCP/IP) `NetStation` talks to an EGI / Philips NetStation EEG host using the ECI (Experiment Control Interface) protocol. Instead of an electrical pulse it sends *named events* — a 4-character code with an onset time, a duration, and optional key/value payloads — directly into the EEG stream, and it starts/stops recording and synchronises the host clock remotely.

```
import "github.com/chrplr/goxpyriment/triggers"

ns, err := triggers.NewNetStation("134.225.198.12") // default ECI port 55513
if err != nil { log.Fatal(err) }
defer func() {
    // Always check this: a failed stop can leave an unreadable .mff on the host.
    if err := ns.Close(); err != nil { log.Printf("netstation: %v", err) }
}()                          // stops recording + disconnects (blocks ~2 s to flush)

ns.Synchronize()             // align the host clock to ours (call once after connect)
ns.StartRecording()

// In the trial loop — send the marker right after the VSYNC flip:
onset, _ := exp.ShowTS(stim)
ns.SendEvent("STIM")         // now, 1 ms, no keys
```

```
// Full form: explicit onset, duration and key/value metadata:
ns.SendEventFull(triggers.Event{
    Code:     "RESP",
    Start:    time.Now(),   // zero value = now; pass the flip time to mark true onset
    Duration: 2 * time.Millisecond,
    Keys:     []triggers.EventKey{{Code: "corr", Value: 1}, {Code: "rt", Value: 423}},
})

ns.StopRecording()
```

Timing note: a network event marker is convenient (it carries a rich label and needs no wiring) but its latency is bounded by the TCP round-trip, not the sub-millisecond edge of a TTL pulse. For hard sub-millisecond EEG synchrony, prefer a `Pulse` on a TTL device for the onset marker and use NetStation events for the accompanying metadata. The driver always advertises the `QNTEL` (Intel/little-endian) handshake and encodes every field little-endian, so it is portable regardless of the machine it runs on. See `tests/test_netstation` for a runnable session.

Never leave a recording open. The output `.mff` is written by NetStation Acquisition — no ECI command names it, chooses its format or finalizes it, so nothing here can change the file's format. What *does* depend on this client is whether the recording is closed properly. A bundle that contains `Acquiring.xml` and no `info.xml`, whose events and EEG signal are both unreadable, is EGI's documented signature of an acquisition that never completed its recording. Two rules follow:

- Check the error from `Close` (as above). It reports a refused stop instead of swallowing it, and every ECI command's acknowledgement is now validated — a host that answers `F` (refused) or `R` (not ready to record) produces an error rather than a silent success.
- Do not call `log.Fatal` between `StartRecording` and `Close`. `log.Fatal` exits without running deferred functions, so the host keeps acquiring. Put the session in a `run() error` function and fail from `main` after the deferred `Close` has run — `tests/test_netstation/main.go` shows the pattern, including a Ctrl-C trap.

If a recording was left open, EGI's File Validator utility (shipped with Net Station 5.4, under Applications ▸ EGI ▸ Utilities) removes `Acquiring.xml` and makes the file openable again.

BEL video recorder (labelled camera over TCP/IP) `VideoRecorder` is the client for the NeuroSpin EEG-room **behavioral video recorder** ("BEL_video"): a camera PC films the participant, burns event labels (subject id, trial, condition, …) into the footage, and saves an AVI. This type starts/stops that recording and pushes the labels over TCP — the capture and encoding stay on the camera PC. It is not an eye tracker: there is no gaze, pupil, or calibration data, only a labelled video.

```
vr, err := triggers.NewVideoRecorder("192.168.8.212") // default port 55113
if err != nil { log.Fatal(err) }
defer vr.Close()            // stops recording + disconnects
```

```
vr.Start()
vr.SetSubject("bb0012025")  // "NIP:<id>" — names the output file
vr.Label("TRL", "001")      // "KEY:VALUE" overlay, shown until the next label / timeou
vr.Label("CND", "007")
vr.Stop()
```

Unlike NetStation, the recorder's protocol has **no framing and no acknowledgement** — it is fire-and-forget, and the server reads one socket buffer per captured video frame. Two messages sent too close together can therefore arrive coalesced and be mis-parsed, so `VideoRecorder` waits a short `SendGap` (default 50 ms; tune with `triggers.WithVRSendGap`, or set 0 to disable) after each message. This makes the labels unsuitable for precise stimulus timing — treat them as monitoring/annotation metadata, not event markers. See `tests/test_videorecorder` for a runnable session.

Device	Type	Role	Connection	Ack?
EGI NetStation	`NetStation`	EEG event markers + recording control + clock sync	TCP/IP (ECI, port 55513)	yes (1 byte/command)
BEL_video	`VideoRecorder`	Labelled video recording control	TCP/IP (port 55113)	no (fire-and-forget)

---

18. Display Compositor Bypass

A compositing window manager (compositor) intercepts every application frame and blends it with other on-screen elements before sending the result to the display. This introduces additional latency and jitter that are problematic for millisecond-accurate stimulus presentation. The degree to which `goxpyriment` can avoid this overhead depends on the operating system.

On **Linux with X11**, SDL3 automatically sets the `_NET_WM_BYPASS_COMPOSITOR` window property when the application enters fullscreen mode. Compliant compositors (KWin, Mutter, Compton) respond by *unredirecting* the fullscreen window — routing its framebuffer directly to the display scan-out pipeline without compositing. The result is presentation latency below 1 ms, comparable to a compositor-free session. On **Linux with Wayland**, the compositor is always the intermediary for frame delivery, but modern compositors (KWin 5.21+, GNOME Mutter 44+) support *direct scan-out* for fullscreen applications, achieving similar low-latency behavior in practice. The best achievable timing on Linux is obtained by running without any display server: setting the environment variable `SDL_VIDEODRIVER=kmsdrm` before launching the experiment directs SDL3 to communicate with the Linux KMS/DRM subsystem directly, bypassing X11 and Wayland entirely. This configuration — typically used from

a virtual terminal (`Ctrl+Alt+F2`) — is recommended for the most demanding timing requirements such as single-frame subliminal stimulation or high-frequency RSVP.

On **Windows**, SDL3's fullscreen mode creates an exclusive fullscreen surface that bypasses the Desktop Window Manager (DWM), again yielding frame-interval jitter typically below 0.3 ms (one standard deviation), comparable to Linux.

macOS is the exception. Apple's Metal rendering pipeline always delivers frames to the display through the WindowServer compositor, and no third-party application can bypass it — even in fullscreen.

These platform differences can be quantified empirically with the `display` and `av` sub-tests of the bundled `Timing-Tests` suite.

---

19. Variable Refresh Rate (VRR) Stimulus Presentation

Standard displays refresh at a fixed rate (e.g., 60 Hz), which quantises every stimulus duration to multiples of the frame period (16.67 ms at 60 Hz). A researcher wishing to present a stimulus for 10 ms, 17 ms, or 23 ms cannot do so on such a display without relying on temporal dithering or hardware frame-blending techniques that introduce their own artefacts. Variable Refresh Rate (VRR) technology — marketed as AMD FreeSync, NVIDIA G-Sync, or the VESA Adaptive-Sync standard — removes this constraint by allowing the display to hold each frame for an arbitrary duration and refresh only when instructed to do so by the GPU.

`goxpyriment` exposes VRR through a single API call: `exp.Screen.SetVSync(0)` disables the VSYNC block, so that every subsequent `SDL_RenderPresent` call returns immediately. The following pattern then achieves arbitrary stimulus durations with duration error typically below 0.5 ms (one standard deviation):

1. Draw the stimulus and call `screen.FlipTS()`, which presents the frame and records `onsetNS` (an SDL nanosecond timestamp) immediately after `Present()` returns.
2. Busy-wait for the desired duration d: spin until `time.Now() >= onset + d`. The last ~500 µs use a CPU spin loop (already used in the trigger timing routines) to eliminate OS scheduling jitter.
3. Draw the blank screen, call `FlipTS` again to record `offsetNS`.

The actual software-controlled duration is `(offsetNS - onsetNS) / 1e6` ms. On a VRR-capable display this equals the true on-screen duration plus a small, constant hardware pipeline latency (GPU scan-out and panel response, typically 1–5 ms) that can be calibrated once with a photodiode using the `frames` timing sub-test of the `Timing-Tests` suite.

Every VRR panel supports VRR only within a finite refresh-rate window (e.g., 48–144 Hz, corresponding to frame durations of 6.9–20.8 ms for a typical gaming monitor). Durations outside this window cause the panel to revert to fixed-rate behaviour, re-introducing quantisation. The `Timing-Tests` suite includes a dedicated `vrr` sub-test that sweeps target durations from 1 ms to a user-specified maximum in 1 ms steps

and reports duration errors at each step, directly revealing the panel's VRR window in the output data.

On a display without VRR support, disabling VSYNC produces frame tearing. The `vrr` sub-test detects this automatically: on a non-VRR display the duration errors cluster at multiples of the frame period rather than being uniformly small, providing an unambiguous diagnostic.

---

20. Putting It All Together

Here is a skeleton that illustrates how the concepts compose in a realistic experiment:

```
package main

import (
    "fmt"
    "log"

    "github.com/chrplr/goxpyriment/control"
    "github.com/chrplr/goxpyriment/design"
    "github.com/chrplr/goxpyriment/stimuli"
)

const (
    NReps           = 10
    FixationDuration = 500
    ResponseTimeout  = 3000
)

type trialDef struct {
    word  string
    color control.Color
    name  string
}

func main() {
    exp := control.NewExperimentFromFlags("Color Word Task", control.Black, control.Whi
    defer exp.End()

    exp.AddDataVariableNames([]string{"trial", "word", "ink", "key", "rt_ms", "correct"

    // Build trial list
    words  := []string{"RED", "GREEN", "BLUE"}
    colors := []control.Color{control.Red, control.Green, control.Blue}
    names  := []string{"red", "green", "blue"}
    keys   := []control.Keycode{control.K_R, control.K_G, control.K_B}
```

```
    var trials []trialDef
    for _, word := range words {
        for j := range colors {
            trials = append(trials, trialDef{word, colors[j], names[j]})
        }
    }
    for i := 0; i < NReps-1; i++ {
        trials = append(trials, trials[:9]...)
    }
    design.ShuffleList(trials)

    // Preload stimuli
    fixation := stimuli.NewFixCross(20, 2, control.White)

    err := exp.Run(func() error {
        // Preload fixation cross (it will be used every trial)
        stimuli.PreloadVisualOnScreen(exp.Screen, fixation)

        exp.ShowInstructions(
            "Name the INK COLOR of each word.\n\n" +
            "R = Red   G = Green   B = Blue\n\n" +
            "Press SPACE to start.",
        )

        for i, t := range trials {
            // Fixation
            exp.ShowTimed(fixation, FixationDuration)

            // Stimulus + Response
            stim := stimuli.NewTextLine(t.word, 0, 0, t.color)
            key, rt, _ := exp.ShowAndGetRT(stim, keys, ResponseTimeout)

            // Score
            correct := false
            for j, k := range keys {
                if key == k && names[j] == t.name {
                    correct = true
                    break
                }
            }

            exp.Data.Add(i, t.word, t.name, key, rt, correct)
            fmt.Printf("trial %3d  %s/%s  rt=%dms  %v\n", i, t.word, t.name, rt, correc

            if !correct {
                exp.Audio.PlayBuzzer()
            }
```

```
            // Release texture (new TextLine each trial)
            stim.Unload()

            exp.Blank(500)
        }

        return control.EndLoop
    })

    if err != nil && !control.IsEndLoop(err) {
        log.Fatalf("experiment error: %v", err)
    }
}
```

Key patterns illustrated:

- `NewExperimentFromFlags` + `defer exp.End()` at the top.
- `exp.AddDataVariableNames` before the run loop.
- `design.ShuffleList` for trial randomization.
- `stimuli.PreloadVisualOnScreen` inside `exp.Run` for timing-sensitive stimuli.
- `exp.Show` for single stimuli; `exp.Blank` for ITIs.
- `exp.Keyboard.WaitKeysRT` for response + RT.
- `stim.Unload()` for dynamically created stimuli.
- `exp.Audio.PlayBuzzer()` for feedback.
- `return control.EndLoop` at the end.

For the complete function signatures and type definitions, see the API Reference. For more worked examples, browse the `examples/` directory.